SIEMENS

TIA Portal Openness API 1

Export/import 2

TIA Portal Openness: API for automation of engineering workflows

Manual

21.00.00.00

03/2026

NOTICE

This document was downloaded from the following site:

https://docs.tia.siemens.cloud

Siemens assumes no liability for the accuracy, completeness, or timeliness of the data provided.

Use is at your own risk.

Legal information

Warning notice system

This manual contains notices you have to observe in order to ensure your personal safety, as well as to prevent damage to property. The notices referring to your personal safety are highlighted in the manual by a safety alert symbol, notices referring only to property damage have no safety alert symbol. These notices shown below are graded according to the degree of danger.

DANGER

indicates that death or severe personal injury **will** result if proper precautions are not taken.

WARNING

indicates that death or severe personal injury **may** result if proper precautions are not taken.

CAUTION

indicates that minor personal injury can result if proper precautions are not taken.

NOTICE

indicates that property damage can result if proper precautions are not taken.

If more than one degree of danger is present, the warning notice representing the highest degree of danger will be used. A notice warning of injury to persons with a safety alert symbol may also include a warning relating to property damage.

Qualified Personnel

The product/system described in this documentation may be operated only by **personnel qualified** for the specific task in accordance with the relevant documentation, in particular its warning notices and safety instructions. Qualified personnel are those who, based on their training and experience, are capable of identifying risks and avoiding potential hazards when working with these products/systems.

Proper use of Siemens products

Note the following:

WARNING

Siemens products may only be used for the applications described in the catalog and in the relevant technical documentation. If products and components from other manufacturers are used, these must be recommended or approved by Siemens. Proper transport, storage, installation, assembly, commissioning, operation and maintenance are required to ensure that the products operate safely and without any problems. The permissible ambient conditions must be complied with. The information in the relevant documentation must be observed.

Trademarks

All names identified by ® are registered trademarks of Siemens Aktiengesellschaft. The remaining trademarks in this publication may be trademarks whose use by third parties for their own purposes could violate the rights of the owner.

Disclaimer of Liability

We have reviewed the contents of this publication to ensure consistency with the hardware and software described. Since variance cannot be precluded entirely, we cannot guarantee full consistency. However, the information in this publication is reviewed regularly and any necessary corrections are included in subsequent editions.

Siemens Aktiengesellschaft
Digital Industries
Postfach 48 48
90026 NÜRNBERG
GERMANY

03/2026 Subject to change

Copyright © Siemens 2026.
All rights reserved

Table of contents

1 TIA Portal Openness API 19

1.1 TIA Portal Openness object 19
1.1.1 TIA Portal Openness object model 19
1.1.2 Blocks and types of the TIA Portal Openness object model 23
1.1.3 Hierarchy of hardware objects of the object model 28
1.1.4 Object list 30
1.2 General functions 36
1.2.1 Introduction 37
1.2.2 Information about installed TIA Portal Openness versions 37
1.2.3 TIA Portal Openness IntelliSense support 38
1.2.4 Standard libraries 39
1.2.5 Use of the code examples 40
1.2.6 Example program 43
1.2.7 Programming steps 47
1.2.8 Connecting to the TIA Portal 48
1.2.9 TIA Portal Openness firewall 53
1.2.10 Event handlers 55
1.2.11 Program-controlled acknowledgement of dialogs with system events 58
1.2.12 Terminating the connection to the TIA Portal 61
1.2.13 Diagnostic interfaces on TIA Portal 62
1.2.14 Exclusive access 69
1.2.15 Dynamic behaviours 71
1.2.16 Working with associations 76
1.2.17 Working with compositions 77
1.2.18 Verifying object equality 78
1.2.19 Read operations for attributes 79
1.2.20 Transaction handling 0
1.2.21 Creating a DirectoryInfo/FileInfo object 0
1.2.22 Self-description support for attributes, navigators, actions, and services 0
1.2.23 Setting attributes of Blocks, DBs & UDTs 0
1.2.24 Notes on performance of TIA Portal Openness 0
1.3 Functions for projects and project data 0
1.3.1 Opening a project 0
1.3.2 Creating a project 0
1.3.3 Accessing general settings of the TIA Portal 0
1.3.4 Archiving and Retrieving a project 0
1.3.5 Accessing read-only TIA Portal project 0
1.3.6 Accessing languages 0
1.3.7 Determining the object structure and attributes 0

1.3.8 Access software target 0
1.3.9 Accessing and enumerating multilingual texts 0
1.3.10 Updating project properties 0
1.3.11 Read project related attributes 0
1.3.12 Deleting project graphics 0
1.3.13 Compiling a project 0
1.3.14 Modifying project in UMAC 0
1.3.15 Saving a project 0
1.3.16 Closing a project 0
1.3.17 Export/Import system diagnostics settings 0
1.3.18 Identifying cross session object for Hardware & Software 0
1.3.19 Managing exception for missing component in project 0
1.3.20 Accessing multilingual support 0
1.4 Functions for accessing Teamcenter Gateway 0
1.4.1 Connecting to the Teamcenter Gateway 0
1.4.2 Disconnecting to the Teamcenter Gateway 0
1.4.3 Connecting to the ConnectSSO 0
1.4.4 Saving Project/Global library in Teamcenter 0
1.4.5 Saving Project/Global library with proxy object 0
1.4.6 Saving Project/Global library with new item 0
1.4.7 Accessing custom attributes in Teamcenter 0
1.4.8 Saving Project to Teamcenter as new item 0
1.4.9 Saving project using save to item with proxy object 0
1.4.10 Saving Project as new item with proxy object 0
1.4.11 Saving project using save as new revision with proxy object 0
1.4.12 Saving Project to Teamcenter as new revision 0
1.4.13 Downloading Project/Global library from Teamcenter 0
1.4.14 Searching Project / Global library from Teamcenter 0
1.4.15 Checkout Project/Global library dataset in Teamcenter 0
1.4.16 Checkin Project/Global library dataset in Teamcenter 0
1.4.17 Cancel checkout dataset in Teamcenter 0
1.5 Functions for Connections 0
1.5.1 Configurable attributes of a port-to-port connection 0
1.6 Functions on libraries 0
1.6.1 Functions for objects and instances 0
1.6.2 Accessing global libraries 0
1.6.3 Accessing library base type 0
1.6.4 Accessing global library languages 0
1.6.5 Opening libraries 0
1.6.6 Comparing libraries 0
1.6.7 Enumerating open libraries 0
1.6.8 Saving and closing libraries 0
1.6.9 Archiving and retrieving a library 0
1.6.10 Creating global libraries 0
1.6.11 Accessing folders in a library 0
1.6.12 Accessing types 0

1.6.13 Accessing type versions 0
1.6.14 Accessing blocks located in library 0
1.6.15 Accessing instances 0
1.6.16 Accessing master copies 0
1.6.17 Create master copy from a project in library 0
1.6.18 Cleaning up library 0
1.6.19 Create an object from a master copy 0
1.6.20 Copying master copies 0
1.6.21 Creating new type from document 0
1.6.22 Updating type from document 0
1.6.23 Determining out-of-date type instances 0
1.6.24 Updating the project 0
1.6.25 Updating a library 0
1.6.26 Cleaning up library 0
1.6.27 Harmonize project from project library 0
1.6.28 Updating Structure during Library Update 0
1.6.29 Updating libraries type 0
1.6.30 Deleting library content 0
1.6.31 Setting default version of a type 0
1.6.32 Getting default version of a type 0
1.6.33 Improved types consistency state 0
1.6.34 Editing library type version 0
1.6.35 Discarding library type version 0
1.6.36 Managing alarm text list in PLC and Mastercopy library 0
1.6.37 Managing DoNotUse in Library 0
1.6.38 Exporting library type version as document 0
1.6.39 Retrieving supported export formats for library 0
1.6.40 Releasing library type version 0
1.7 Functions for accessing devices, networks and connections 0
1.7.1 Open the 'Devices & networks' editor 0
1.7.2 Querying PLC and HMI targets 0
1.7.3 Accessing attributes of an address object 0
1.7.4 Accessing the channels of a module 0
1.8 Functions on networks 0
1.8.1 Creating a subnet 0
1.8.2 Accessing subnets 0
1.8.3 Accessing internal subnets 0
1.8.4 Get type identifier of subnets 0
1.8.5 Accessing attributes of a subnet 0
1.8.6 Deleting a global subnet 0
1.8.7 Enumerate all participants of a subnet 0
1.8.8 Enumerate IO systems of a subnet 0
1.8.9 Accessing nodes 0
1.8.10 Accessing attributes of a node 0
1.8.11 Connecting a node to a subnet 0
1.8.12 Disconnect a node from a subnet 0

1.8.13 Creating an IO system 0
1.8.14 Accessing the attributes of an IO system 0
1.8.15 Connecting an IO connector to an IO system 0
1.8.16 Get master system or IO system of an interface 0
1.8.17 Get an IO Controller 0
1.8.18 Get an IO Connector 0
1.8.19 Disconnecting an IO connector from an IO system or a DP mastersystem 0
1.8.20 Accessing attributes of a DP mastersystem 0
1.8.21 Accessing attributes of PN Driver 0
1.8.22 Accessing attributes of a profinet io system 0
1.8.23 Deleting a DP master system 0
1.8.24 Deleting a profinet io system 0
1.8.25 Creating a DP master system 0
1.8.26 Accessing port interconnection information of port device item 0
1.8.27 Attributes of port inter-connection 0
1.8.28 Accessing the attributes of a port 0
1.8.29 Enumerate DP master systems of a subnet 0
1.8.30 Enumerate assigned IO connectors 0
1.8.31 Connecting a DP IO connector to a DP master system 0
1.8.32 Accessing AS-i profile and parameter attributes for virtual slaves 0
1.8.33 Accessing CM DP as DP slave and transfer area 0
1.8.34 Coupling 1516 isochron central and decentral Profinet 0
1.8.35 Openness for CP 1604/CP 1616/CP 1626 0
1.8.36 Openness transfer areas for PnPn coupler 0
1.8.37 Openness virtual modules/submodules for ET 200SP PN HF 0
1.8.38 Accessing domain settings 0
1.8.39 Accessing redundancy mode attribute 0
1.8.40 Creating CCDX transfer area 0
1.8.41 Deleting CCDX transfer area 0
1.8.42 Modifying CCDX transfer areas configuration 0
1.9 Functions on devices 0
1.9.1 Mandatory attributes of devices 0
1.9.2 Get type identifier of devices and device items 0
1.9.3 Set App ID on device and device Items 0
1.9.4 Get App ID on device and device Items 0
1.9.5 Remove App ID on device and device items 0
1.9.6 Creating a device 0
1.9.7 Enumerating devices 0
1.9.8 Accessing devices 0
1.9.9 Accessing the TIA Portal hardware catalog 0
1.9.10 Deleting a device 0
1.9.11 Bulk changing hardware parameters 0
1.10 Functions on device items 0
1.10.1 Mandatory attributes of device items 0
1.10.2 Creating and plugging a device item 0
1.10.3 Moving device items into another slot 0

1.10.4 Copying a device item 0
1.10.5 Setting user defined logo settings 0
1.10.6 Deleting a device item 0
1.10.7 Enumerate device items 0
1.10.8 Accessing device items 0
1.10.9 Accessing device item as interface 0
1.10.10 Accessing change device 0
1.10.11 Accessing attributes of an I/O device interface 0
1.10.12 Getting subnet of device item 0
1.10.13 Accessing attributes of IoController 0
1.10.14 Accessing attributes of IoConnector 0
1.10.15 Accessing address controller 0
1.10.16 Accessing addresses 0
1.10.17 Accessing hardware identifiers 0
1.10.18 Accessing hardware identifier controller 0
1.10.19 Accessing channels of device items 0
1.10.20 Accessing normalized type identifiers 0
1.10.21 Creating and exporting psc file 0
1.10.22 Connection handling for extension racks 0
1.10.23 Telecontrol management datapoint 0
1.11 Functions for accessing the data of a PLC device 0
1.11.1 Functions for downloading data to PLC device 0
1.11.1.1 Accessing system diagnostics properties for S7-1200/1500 PLCs 0
1.11.1.2 Downloading PC System 0
1.11.1.3 Downloading to PLC devices 0
1.11.1.4 Downloading PLC to a Windows folder 0
1.11.1.5 Downloading User Management data 0
1.11.1.6 Downloading Master Secret to PLC 0
1.11.1.7 Downloading to PLC with VoT 0
1.11.1.8 Managing PLC Master Secret in PLCs 0
1.11.1.9 Setting / deleting a PLC Master Secret in a PLCs 0
1.11.1.10 Running and stopping PLC 0
1.11.1.11 Supporting callbacks 0
1.11.1.12 Protecting PLC through password 0
1.11.1.13 Handling PLC block binding passwords 0
1.11.1.14 Uploading PLC device 0
1.11.1.15 Comparing PLC software 0
1.11.1.16 Updating PLC program 0
1.11.1.17 Comparing PLC hardware 0
1.11.1.18 Setting password policy for PLC 0
1.11.1.19 Managing dynamic certificate settings 0
1.11.1.20 Managing runtime account locking configuration 0
1.11.2 Functions for accessing PLC service 0
1.11.2.1 Access level setting 0
1.11.2.2 Accessing Software Checksum 0
1.11.2.3 Assigning PC interface 0

1.11.2.4 Setting PLC priority and resource assignment export 0
1.11.2.5 Determining the status of a PLC 0
1.11.2.6 Setting PLC Online 0
1.11.2.7 Accessing parameters of an online connection 0
1.11.2.8 Accessing Configuration accessible devices 0
1.11.2.9 Accessing fingerprint for quick station compare 0
1.11.2.10 Accessing Cross Reference Service on Step7 0
1.11.2.11 Getting Cross References for Step7 0
1.11.2.12 Setting PLC online of R/H system 0
1.11.2.13 Accessing software container from primary PLC of R/H system 0
1.11.2.14 Downloading PLCs of R/H System 0
1.11.2.15 Establishing or disconnecting the online connection to the PLC 0
1.11.2.16 Assigning project language to PLC 0
1.11.2.17 Assigning watch & force tables for web server and PLC display 0
1.11.2.18 Managing certificate 0
1.11.2.19 Managing Web Application Configuration 0
1.11.2.20 Managing system web page 0
1.11.2.21 Managing Webserver user of SIWAREX modules 0
1.11.2.22 Managing Syslog configuration 0
1.11.2.23 Supporting secure S7 communication TLS 0
1.11.2.24 Updating module description 0
1.11.2.25 Supporting UMAC on PLC device 0
1.11.2.26 Supporting IP Accessibility 0
1.11.2.27 Accessing PLC access control configuration provider 0
1.11.2.28 Setting a display password 0
1.11.2.29 Accessing web server and OPC UA user management 0
1.11.3 Blocks 0
1.11.3.1 Querying the 'Program blocks' group 0
1.11.3.2 Querying the system group for system blocks 0
1.11.3.3 Enumerating system subgroups 0
1.11.3.4 Enumerating user-defined block groups 0
1.11.3.5 Enumerating all blocks 0
1.11.3.6 Querying information of a block/user data type 0
1.11.3.7 Setting and removing protections from a block 0
1.11.3.8 Setting up write protection of blocks 0
1.11.3.9 Deleting block 0
1.11.3.10 Accessing Data blocks 0
1.11.3.11 Creating and Renaming group for blocks 0
1.11.3.12 Deleting group for blocks 0
1.11.3.13 Accessing attributes of all blocks 0
1.11.3.14 Creating a ProDiag-FB 0
1.11.3.15 Accessing supervisions and properties of ProDiag-FB 0
1.11.3.16 Assigning ProDiag FB for DBs & Tag Table 0
1.11.3.17 Reading ProDiag-FB blocks and attributes 0
1.11.3.18 Exporting ProDiag alarm message 0
1.11.3.19 Adding an external file 0

1.11.3.20 Generate source from block 0
1.11.3.21 Generating blocks from source 0
1.11.3.22 Generating from source of known source format 0
1.11.3.23 Deleting user data type 0
1.11.3.24 Accessing PLC system data types 0
1.11.3.25 Deleting an external file 0
1.11.3.26 Starting the block editor 0
1.11.3.27 Changing blocks using fingerprints 0
1.11.3.28 Generating/deleting blocks for user defined pages 0
1.11.3.29 Writing access for OB block priority attribute 0
1.11.3.30 Generating block/UDT from external source file in specific user group 0
1.11.3.31 Generating loadable file for blocks outside of software units 0
1.11.4 Tags and Tag tables 0
1.11.4.1 Starting the 'PLC Tags' editor 0
1.11.4.2 Querying system groups for PLC tags 0
1.11.4.3 Creating PLC tag table 0
1.11.4.4 Enumerating user-defined groups for PLC tags 0
1.11.4.5 Creating user-defined groups for PLC tags 0
1.11.4.6 Deleting user-defined groups for PLC tags 0
1.11.4.7 Enumerating PLC tag tables in a folder 0
1.11.4.8 Querying information from a PLC tag table 0
1.11.4.9 Reading the time of the last changes of a PLC tag table 0
1.11.4.10 Deleting a PLC tag table from a group 0
1.11.4.11 Enumerating PLC tags 0
1.11.4.12 Accessing PLC tags 0
1.11.4.13 Accessing PLC constants 0
1.11.5 Functions for software units 0
1.11.5.1 Accessing software unit 0
1.11.5.2 Working with software unit 0
1.11.5.3 Accessing software unit underlying objects 0
1.11.5.4 Accessing to existing relations of a unit 0
1.11.5.5 Updating software unit properties 0
1.11.5.6 Publishing software unit object 0
1.11.5.7 Adding external sources in units 0
1.11.5.8 Units as mastercopies 0
1.11.5.9 Updating existing relations & create/delete relations 0
1.11.5.10 Accessing name value type document 0
1.11.5.11 Accessing namespaces for software units 0
1.11.5.12 Accessing namespaces for program blocks 0
1.11.5.13 Accessing namespaces for UDT 0
1.11.5.14 Generating loadable file for software units 0
1.11.6 Functions for fail-safe unit 0
1.11.6.1 Generating fail-safe unit 0
1.11.6.2 Accessing the SafetyUnit 0
1.11.6.3 Creating/deleting SafetyUnit relations 0
1.11.6.4 Creating/Accessing SafetyUnit elements/objects 0

1.11.6.5 Exporting/Importing supervisions under SafetyUnit 0
1.11.6.6 Publishing blocks under the SafetyUnit 0
1.12 Functions for accessing the data of an HMI device 0
1.12.1 Screens 0
1.12.1.1 Creating user-defined screen folders 0
1.12.1.2 Deleting a screen from a folder 0
1.12.1.3 Deleting a screen template from a folder 0
1.12.1.4 Deleting all screens from a folder 0
1.12.2 Cycles 0
1.12.2.1 Deleting a cycle 0
1.12.3 Text lists 0
1.12.3.1 Deleting a text list 0
1.12.4 Graphic lists 0
1.12.4.1 Deleting a graphic list 0
1.12.5 Connections 0
1.12.5.1 Deleting a connection 0
1.12.6 Tag table 0
1.12.6.1 Creating user-defined folders for HMI tags 0
1.12.6.2 Enumerating tags of an HMI tag table 0
1.12.6.3 Deleting an individual tag from an HMI tag table 0
1.12.6.4 Deleting a tag table from a folder 0
1.12.7 VB scripts 0
1.12.7.1 Creating user-defined script folders 0
1.12.7.2 Deleting a VB script from a folder 0
1.12.7.3 Deleting a user-defined folder of an HMI device 0
1.13 Functions for Version Control Interface 0
1.13.1 Accessing VCI system group in project 0
1.13.2 Enumerating user groups in VCI group 0
1.13.3 Accessing VCI Workspace with library properties 0
1.13.4 Enumerating VCI mapped objects in VCI workspace 0
1.13.5 Creating VCI user group in VCI group 0
1.13.6 Connecting object 0
1.13.7 Updating VCI group properties 0
1.13.8 Deleting VCI user group 0
1.13.9 Enumerating VCI workspaces in VCI group 0
1.13.10 Creating VCI workspace in VCI group 0
1.13.11 Updating VCI workspace properties 0
1.13.12 Deleting VCI workspace 0
1.13.13 Exporting individual object to workspace 0
1.13.14 Getting supported file formats 0
1.13.15 Getting VCI workspace mapping properties 0
1.13.16 Deleting mapped object 0
1.13.17 Synchronizing mapped object 0
1.13.18 Getting status of mapped object 0
1.13.19 Getting child status of a mapped hierarchical object 0
1.13.20 Accessing synchronization status of child objects 0

1.13.21 Creating VCI workspace with workspace language 0
1.13.22 Initializing VCI object status 0
1.13.23 Importing Simatic ML with inactive culture 0
1.14 Functions support for Multiuser 0
1.14.1 Creating project server connection 0
1.14.2 Adding project to server 0
1.14.3 Editing project server connection 0
1.14.4 Deleting project server connection 0
1.14.5 Getting project server connection from TIA Portal Setting 0
1.14.6 Getting project server group 0
1.14.7 Adding a project to server 0
1.14.8 Getting server projects 0
1.14.9 Getting available local sessions 0
1.14.10 Creating a local session 0
1.14.11 Saving a local session 0
1.14.12 Deleting local session from server 0
1.14.13 Opening local/exclusive session 0
1.14.14 Opening server project 0
1.14.15 Checking marked objects 0
1.14.16 Closing local session 0
1.14.17 Commiting server project changes 0
1.14.18 Getting lockstate provider 0
1.14.19 Checking project locked 0
1.14.20 Getting lock owner 0
1.14.21 Checking session UptoDate 0
1.14.22 Getting markings from local session 0
1.15 Functions for accessing data of the user management 0
1.15.1 Functions for accessing user, roles and function rights 0
1.15.1.1 Protecting Project 0
1.15.1.2 Engineering Function Rights 0
1.15.1.3 Device Function Rights 0
1.15.1.4 System Roles 0
1.15.1.5 Custom Roles 0
1.15.1.6 Project Users 0
1.15.2 Functions for UMAC Global Users and UMC Server 0
1.15.2.1 Authentication to connect to a UMC Server 0
1.15.2.2 Retrieving an UMC User Group from a UMC Server 0
1.15.2.3 Retrieving an UMC User from a UMC Server 0
1.15.2.4 Adding an UMC User Group 0
1.15.2.5 Getting all UMC groups in TIA Portal 0
1.15.2.6 Adding an UMC User to TIA Portal 0
1.15.2.7 Getting all UMC Users in TIA Portal project 0
1.15.2.8 Finding an UMC User added in TIA Portal 0
1.15.2.9 Finding an UMC User group added in TIA Portal project 0
1.15.2.10 Assigning role to UMC User 0
1.15.2.11 Getting the list of assigned roles of UMC User and UMC User Group. 0

1.15.2.12 Getting status on markable object 0
1.15.2.13 Marking and unmarking objects 0
1.15.2.14 Assigning role to UMC User Group 0
1.15.2.15 Removing role from UMC User and UMC User Group 0
1.15.2.16 Activating/Deactivating UMC User and UMC User Group 0
1.15.2.17 Checking state for UMC User and UMC User Group 0
1.15.2.18 Deleting UMC User and UMC User Group 0
1.15.2.19 Synchronizing UMC user 0
1.15.2.20 Checking UMC data consistency 0
1.15.2.21 Setting runtime session timeout 0
1.15.2.22 Setting alias for local user 0
1.15.2.23 Setting password policies for UMAC 0
1.15.2.24 Accessing Anonymous user in protected project 0
1.15.2.25 Managing offline UMC User And User Groups 0
1.15.2.26 Complete sample code 0
1.16 Functions on OPC 0
1.16.1 Configuring OPC UA server secure communication protocol 0
1.16.2 Setting OPC UA security policy 0
1.16.3 Accessing OPC UA reference namespace 0
1.16.4 Accessing OPC UA server (SIMATIC) interface 0
1.16.5 Accessing OPC UA server interface 0
1.17 Functions for SIMATIC Drive Controller 0
1.17.1 Accessing PLC and X142 channel properties 0
1.17.2 Accessing PROFIdrive Integrated properties 0
1.17.3 Creating SINAMICS Integrated 0
1.17.4 Getting address of central module (X142) 0
1.17.5 Getting address of drive controller telegram 0
1.18 Exceptions 0
1.18.1 Handling exceptions 0
1.19 F-related Openness 0
1.19.1 F-related Openness 0
1.19.2 SafetyBaseIdProvider 0
1.19.3 SafetyModificationsPossible 0
1.19.4 UsernameForFChangeHistory 0
1.19.5 F-Signatures 0
1.19.5.1 F-signatures 0
1.19.5.2 SafetySignatureType 0
1.19.6 SafetySignatureProvider 0
1.19.7 SafetyAdministration 0
1.19.7.1 SafetyAdministration 0
1.19.7.2 IsSafetyOfflineProgramPasswordSet 0
1.19.7.3 SetSafetyOfflineProgramPassword 0
1.19.7.4 RevokeSafetyOfflineProgramPassword 0
1.19.7.5 IsLoggedOnToSafetyOfflineProgram 0
1.19.7.6 LoginToSafetyOfflineProgram 0
1.19.7.7 LogoffFromSafetyOfflineProgram 0

1.19.7.8 Runtime Groups 0
1.19.7.9 Safety settings 0
1.19.8 SafetyPrintout 0
1.19.9 Hardware attributes 0
1.19.9.1 Hardware attributes 0
1.19.9.2 Failsafe_FCapabilityActivated 0
1.20 Functions for Safety Validation Assistant 0
1.20.1 Accessing Safety Validation Assistant 0
1.20.2 Querying the Activation Tests 0
1.20.3 Querying the Evaluation Devices 0
1.20.4 Creating new Activation Tests 0
1.20.5 Duplicating Activation Tests 0
1.20.6 Copying Activation Tests from a library 0
1.20.7 Querying Available Devices of the Activation Test 0
1.20.8 Modifying Activation Tests 0
1.20.9 Overall state of the Activation Tests 0
1.20.10 Validity of the Activation Tests 0
1.20.11 Generating Activation Test reports 0
1.20.12 Exporting Activation Tests 0
1.20.13 Importing Activation Tests 0
1.20.14 Querying the Safety Functions 0
1.20.15 Creating new Safety Functions 0
1.20.16 Duplicating Safety Functions 0
1.20.17 Resetting Test Results of Safety Functions 0
1.20.18 Modifying Safety Functions 0
1.20.19 State of the Safety Functions 0
1.20.20 Validity of the Safety Functions 0
1.20.21 Exporting Safety Functions 0
1.20.22 Importing Safety Functions 0
1.20.23 Querying the Conditions 0
1.20.24 Creating new Conditions 0
1.20.25 Validity of the Conditions 0
1.20.26 Getting the Trace Configuration 0
1.20.27 Modifying Trace Configuration 0
1.20.28 Validity of the Trace Configuration 0

2 Export/import 0
2.1 Overview 0
2.1.1 Basic principles of importing/exporting 0
2.1.2 Field of application for Import/Export 0
2.1.3 Version Specific Simatic ML Import 0
2.1.4 Editing the XML file 0
2.1.5 Exporting configuration data 0
2.1.6 Importing configuration data 0
2.2 Import/export of project data 0
2.2.1 Project graphics 0
2.2.1.1 Exporting/importing graphics 0

2.2.1.2 Exporting all graphics of a project 0
2.2.1.3 Importing graphics to a project 0
2.2.2 Project texts 0
2.2.2.1 Export of project texts 0
2.2.2.2 Import of project texts 0
2.3 Importing/exporting data of an HMI device 0
2.3.1 Importing/exporting data of an HMI device 0
2.3.2 Data structure for import/export 0
2.3.2.1 Structure of an XML file 0
2.3.2.2 Structure of the data for importing/exporting 0
2.3.2.3 Cycles 0
2.3.3 Tag tables 0
2.3.3.1 Exporting HMI tag tables 0
2.3.3.2 Importing HMI tag table 0
2.3.3.3 Exporting an individual tag from an HMI tag table 0
2.3.3.4 Importing an individual tag into an HMI tag table 0
2.3.3.5 Special considerations for the export/import of HMI tags 0
2.3.4 VB scripts 0
2.3.4.1 Exporting VB scripts 0
2.3.4.2 Exporting VB scripts from a folder 0
2.3.4.3 Importing VB scripts 0
2.3.5 Text lists 0
2.3.5.1 Exporting text lists from an HMI device 0
2.3.5.2 Importing a text list into an HMI device 0
2.3.5.3 Advanced XML formats for export/import of text lists 0
2.3.6 Graphic lists 0
2.3.6.1 Exporting graphic lists 0
2.3.6.2 Importing a graphic list 0
2.3.7 Connections 0
2.3.7.1 Exporting connections 0
2.3.7.2 Importing connections 0
2.3.8 Screens 0
2.3.8.1 Overview of exportable screen objects 0
2.3.8.2 Exporting all screens of an HMI device 0
2.3.8.3 Exporting a screen from a screen folder 0
2.3.8.4 Importing screens to an HMI device 0
2.3.8.5 Exporting permanent areas 0
2.3.8.6 Importing permanent areas 0
2.3.8.7 Exporting all screen templates of an HMI device 0
2.3.8.8 Exporting screen templates from a folder 0
2.3.8.9 Importing screen templates 0
2.3.8.10 Exporting a pop-up screen 0
2.3.8.11 Importing a pop-up screen 0
2.3.8.12 Exporting a slide-in screen 0
2.3.8.13 Importing a slide-in screen 0
2.3.8.14 Exporting a screen with a faceplate instance 0

2.3.8.15 Importing a screen with a faceplate instance 0
2.4 Importing/exporting data of a PLC device 0
2.4.1 Blocks 0
2.4.1.1 XML structure of the block interface section 0
2.4.1.2 Changes of the object model and XML file format 0
2.4.1.3 Exporting DBs with snapshots 0
2.4.1.4 Exporting blocks with know-how protection 0
2.4.1.5 Export/Import of SCL blocks 0
2.4.1.6 Export/Import of structured types of SCL blocks 0
2.4.1.7 Export/Import of SCL call blocks 0
2.4.1.8 Export/Import multilingual comments in SCL 0
2.4.1.9 Exporting failsafe blocks 0
2.4.1.10 Exporting system blocks 0
2.4.1.11 Exporting GRAPH blocks with multi-language text 0
2.4.1.12 Importing block 0
2.4.1.13 Exporting blocks 0
2.4.1.14 Importing blocks/UDT with open reference 0
2.4.1.15 Importing blocks/UDT for structural change object 0
2.4.1.16 Export/Import of unit specific publishing attribute of blocks and types 0
2.4.1.17 Creating Instance DB 0
2.4.1.18 Accessing DB value parameter without export 0
2.4.1.19 Export/Import of Plc Alarm TextLists 0
2.4.1.20 Export Document information to SimaticML file 0
2.4.1.21 Export/Import of Alarm Instance Text 0
2.4.1.22 Export/Import of Alarm classes 0
2.4.1.23 Export/Import of global supervision for ProDiag-FB 0
2.4.1.24 Export/import of ProDiag supervisions from Global overview editor 0
2.4.1.25 Export/Import of settings for ProDiag supervisions 0
2.4.1.26 Export/Import Watch & Force Table 0
2.4.1.27 Exporting user data type 0
2.4.1.28 Importing user data type 0
2.4.1.29 Exporting program block as document 0
2.4.1.30 Importing program block from document 0
2.4.1.31 Exporting UDT as document 0
2.4.1.32 Export of data in OPC UA XML format 0
2.4.1.33 Importing UDT from document 0
2.4.1.34 Export/Import of UDT & DB 0
2.4.1.35 Export/Import of Array & Instance DB 0
2.4.2 Tag tables 0
2.4.2.1 Exporting PLC tag tables 0
2.4.2.2 Importing PLC tag table 0
2.4.2.3 Exporting an individual tag or constant from a PLC tag table 0
2.4.2.4 Importing an individual tag or constant into a PLC tag table 0
2.5 Importing/exporting hardware data 0
2.5.1 AML file format 0
2.5.2 Pruned AML 0

2.5.3 Overview of the objects and parameters of the CAx import/export 0
2.5.4 Structure of the CAx data for importing/exporting 0
2.5.5 AML type identifiers 0
2.5.6 Export/Import of base unit Information via AML 0
2.5.7 Export/Import AML with extension rack connection 0
2.5.8 Export/import AML file with GSD/GSDML custom attributes 0
2.5.9 Export/Import AML file with non-GSD/GSDML custom attributes 0
2.5.10 Export/import AML file with pluggable port configuration 0
2.5.11 Export/Import AML file with IO Link 0
2.5.12 Connection handling for extension racks 0
2.5.13 Export/Import AML file with complex tag configuration 0
2.5.14 Export of CAx data 0
2.5.15 Import of CAx data 0
2.5.16 Export/Import of sub modules 0
2.5.17 Export/Import AML file in UMAC environment 0
2.5.18 Export/Import AML file with normalized type identifier 0
2.5.19 Import CAx data without logical address 0
2.5.20 Exceptions during import and export of CAx data 0
2.5.21 Round trip exchange of devices and modules 0
2.5.22 Export/Import topology 0
2.5.23 Import of device with library references 0
2.5.24 Export of a device object 0
2.5.25 Import of a device object 0
2.5.26 Export/Import of device with set address 0
2.5.27 Export/Import of device with channels 0
2.5.28 Export of device item objects 0
2.5.29 Import of device item objects 0
2.5.30 Export/Import of GSD/GSDML based devices and device items 0
2.5.31 Export/Import device configuration with virtual interface 0
2.5.32 Export/Import of subnets 0
2.5.33 Export/Import of IO-systems 0
2.5.34 Export/Import of multilingual comments 0
2.5.35 Export/Import ET200 SP/ET200 AL devices with extension rack connections 0
2.5.36 Export/Import of PLC tags 0
2.5.37 Export/Import of RH/PLC 0
2.5.38 Export/Import AML with customized tag and deviceitems 0
2.5.39 Export/Import of FailSafe PLC 0
2.5.40 Export/Import of Failsafe IO 0
2.5.41 Export/Import vendor specific attribute 0
2.5.42 AML attributes versus TIA Portal Openness attributes 0

TIA Portal Openness API

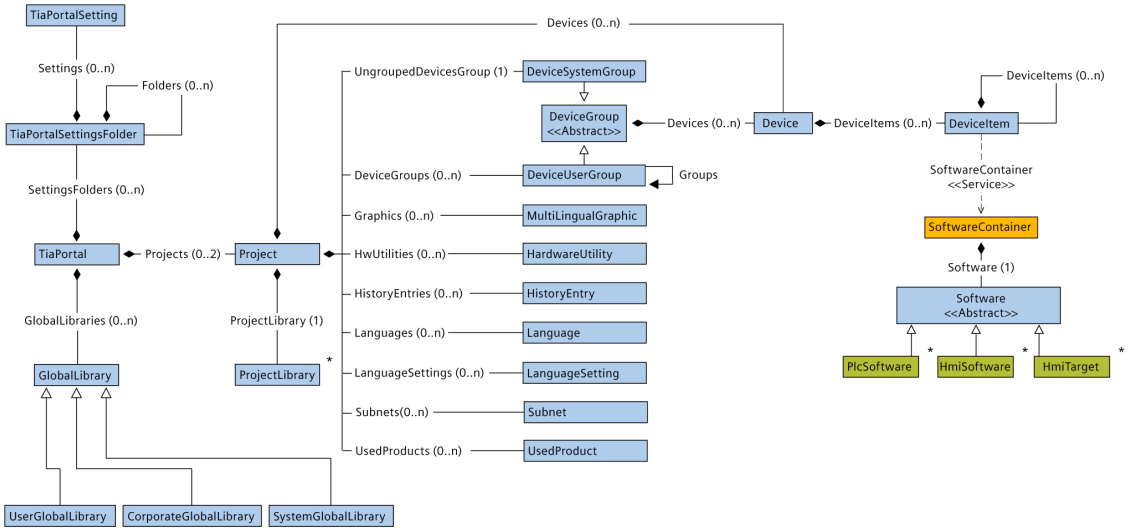
1

1.1 TIA Portal Openness object

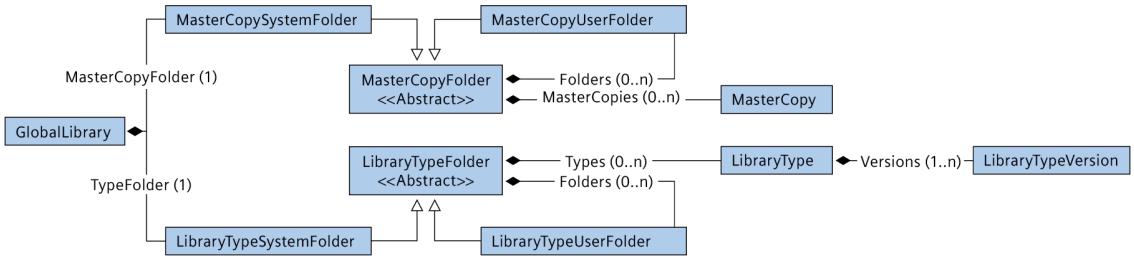
1.1.1 TIA Portal Openness object model

Overview

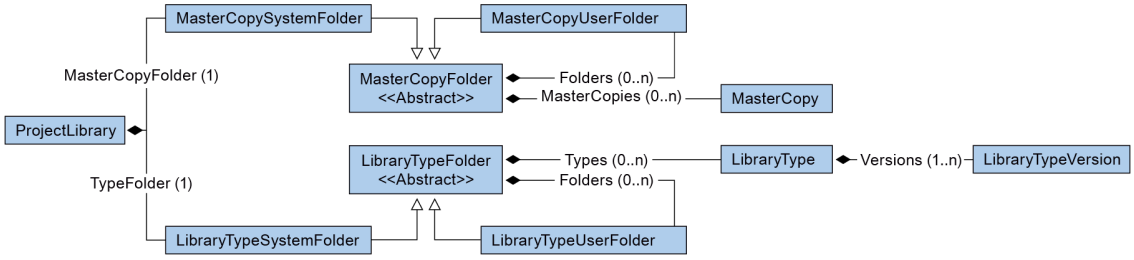
The following diagram describes the highest level of the TIA Portal Openness object model:



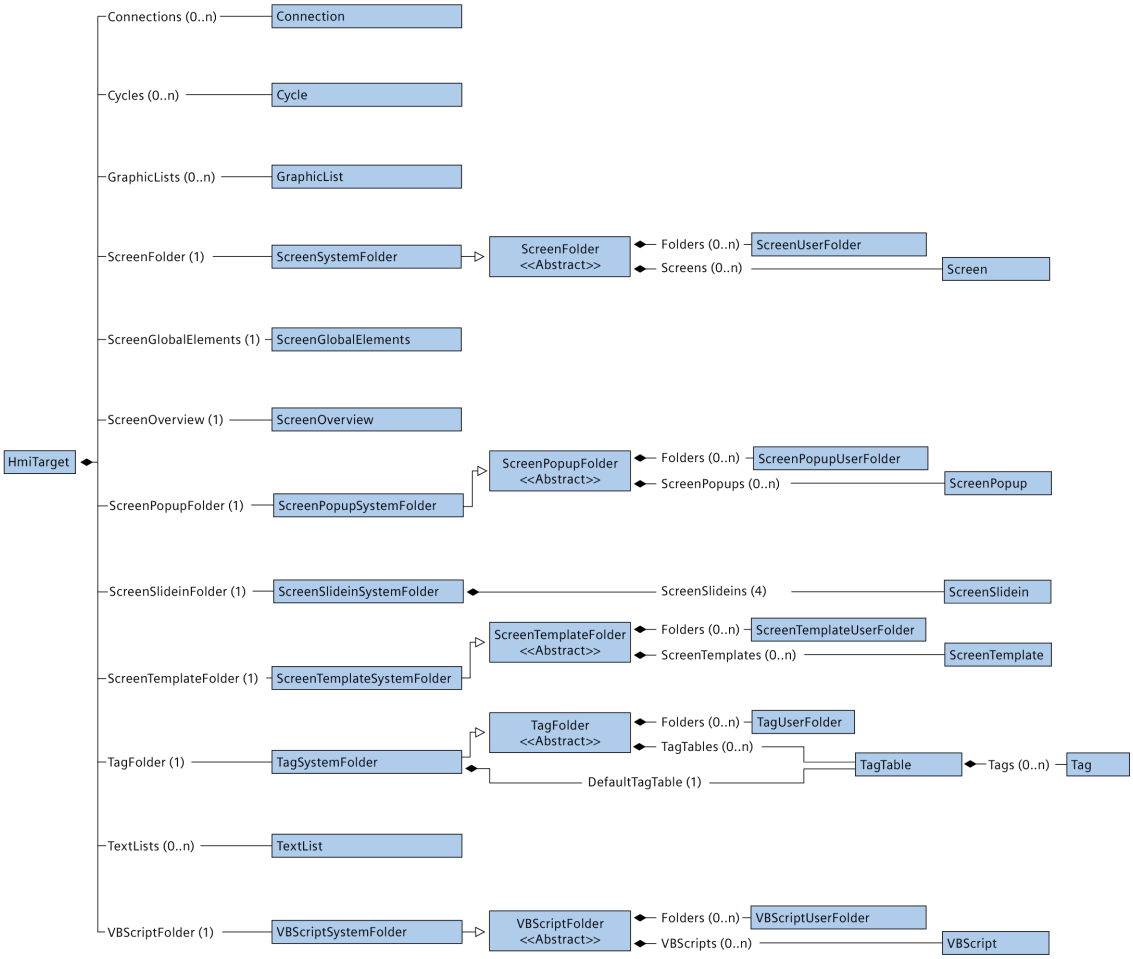
The following diagram describes the objects which are located under GlobalLibrary.



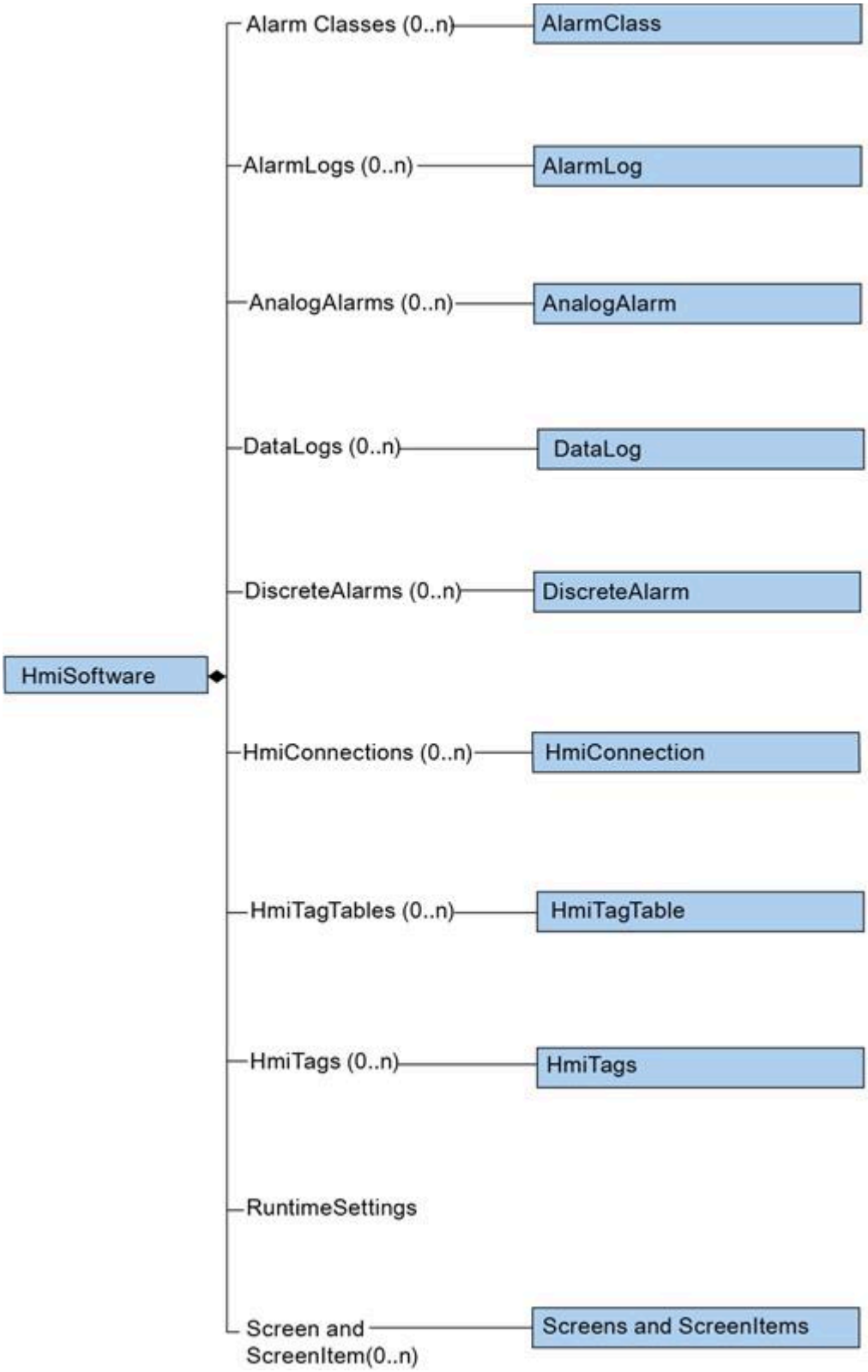
The following diagram describes the objects which are located under ProjectLibrary.



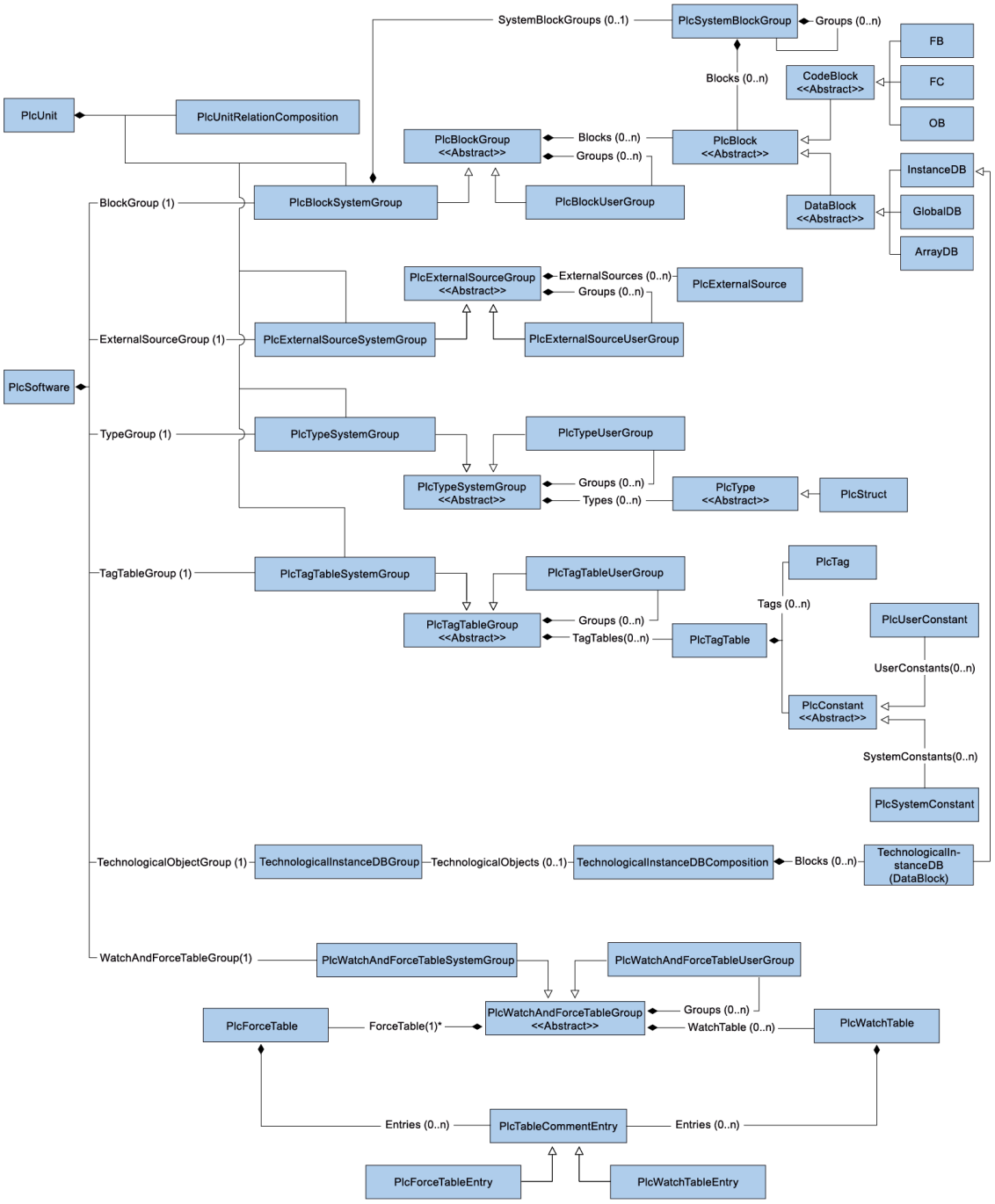
The following diagram describes the objects which are located under HmiTarget.



The following diagram describes the object which are located under HmiSoftware.



The following diagram describes the objects which are located under PlcSoftware.



Note

*The Force Table must be in PlcWatch and ForceTableSystemGroup

Access to objects in lists

You have the following options for addressing an object in a list:

- Address via the index. The counting within the lists starts with `0`.

- Use `Find` method.
  Use this method to address an object via its name. You can use this method for an composition or list. The `Find` method is not recursive.

  Example:

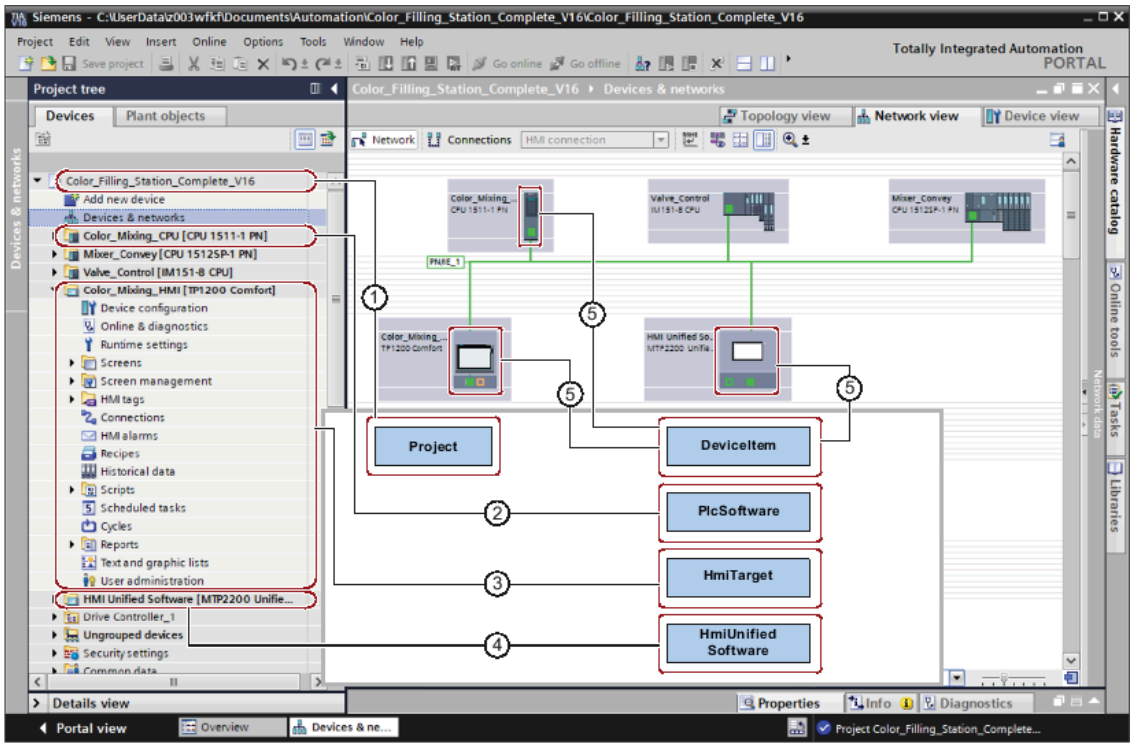
  ```
  ScreenComposition screens = folder.Screens;

  Screen screen = screens.Find("myScreen");
  ```

- Use symbolic names.

Relationship between TIA Portal and TIA Portal Openness object model

The figure below shows the relationship between the object model and a project in the TIA Portal:



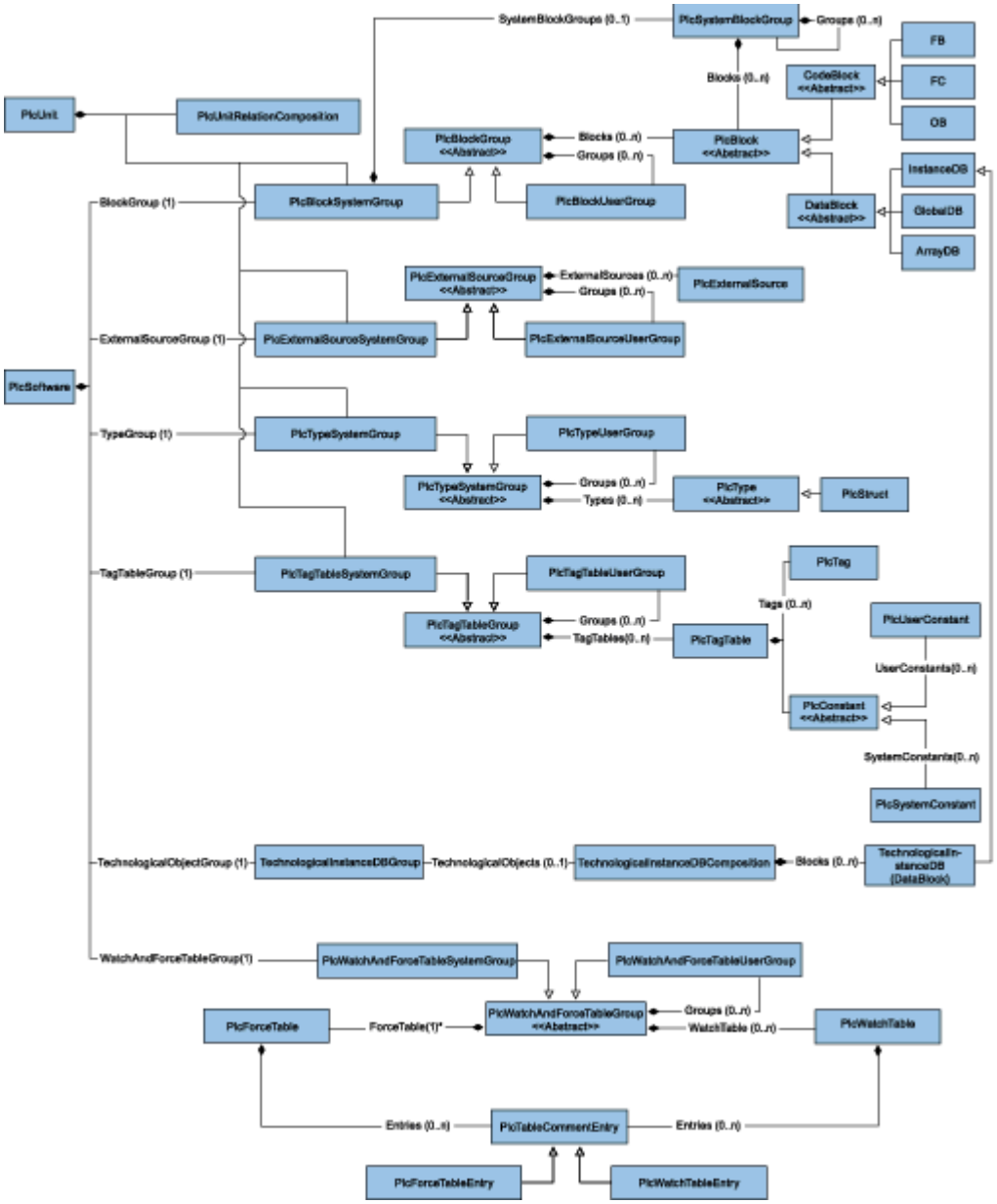
See also

Blocks and types of the TIA Portal Openness object model
Hierarchy of hardware objects of the object model

1.1.2 Blocks and types of the TIA Portal Openness object model

Introduction

The following diagram describes the domain model of the PLCs to give an overview of the current modeling in TIA Portal Openness.



Note

* The Force Table must be in PlcWatch and ForceTableSystemGroup

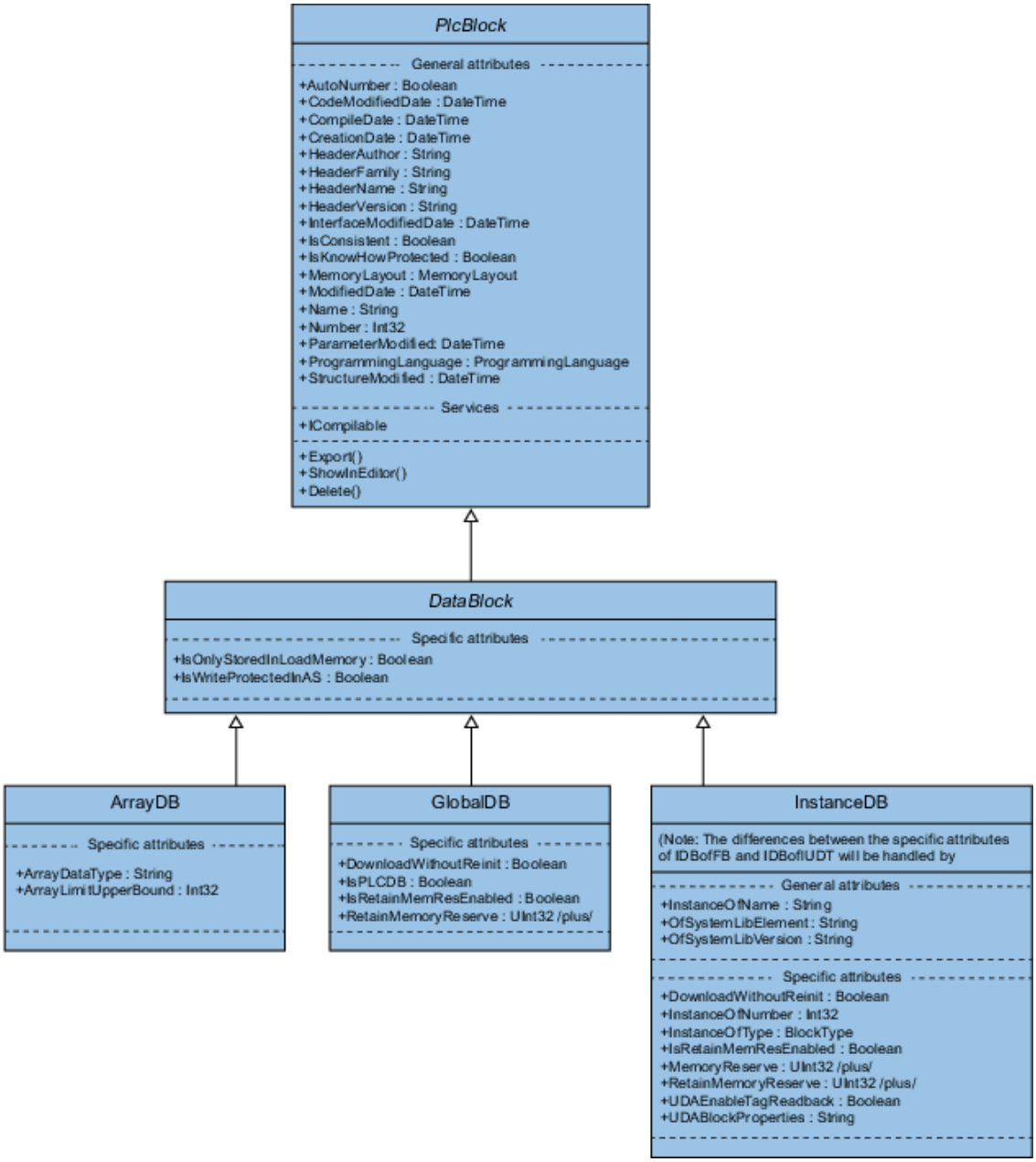
Representation of blocks and types in the TIA Portal Openness API

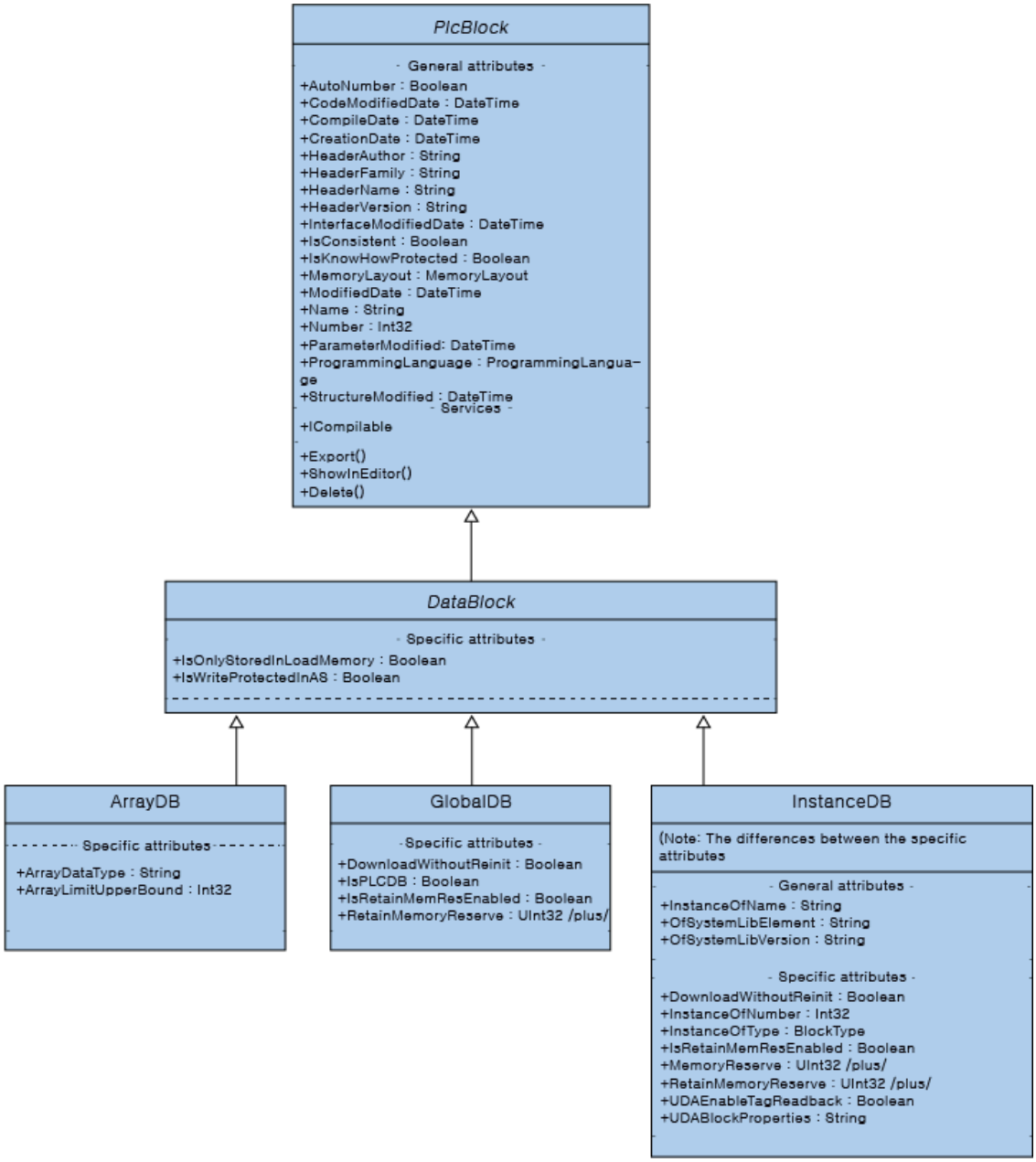
The simplified model part of blocks and for the structure is based on the attributes in the TIA Portal Openness API. These classes provide the export function and for blocks also the compile function.

Class diagrams

In the TIA Portal Openness object model all classes are defined as abstract which aren't directly instantiated.

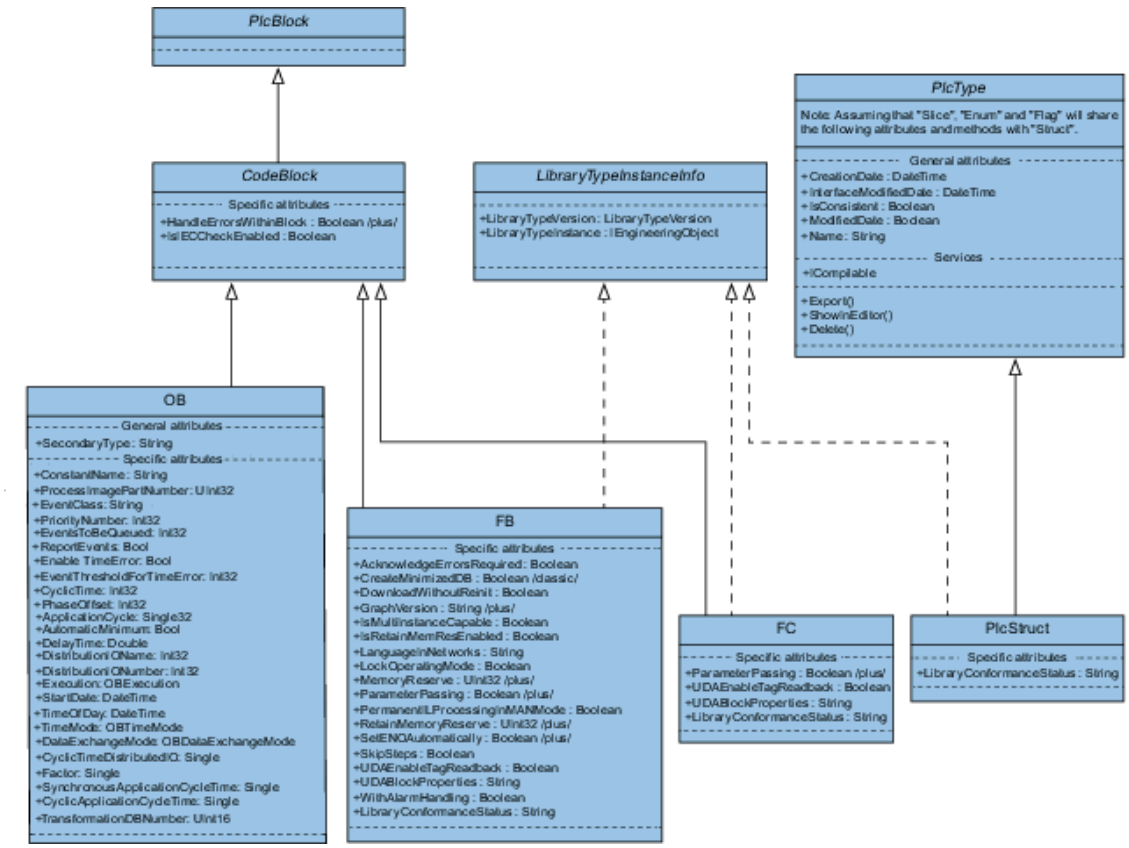
Data


PlcBlock
General attributes
+AutoNumber : Boolean
+CodeModifiedDate : DateTime
+CompileDate : DateTime
+CreationDate : DateTime
+HeaderAuthor : String
+HeaderFamily : String
+HeaderName : String
+HeaderVersion : String
+InterfaceModifiedDate : DateTime
+IsConsistent : Boolean
+IsKnowHowProtected : Boolean
+MemoryLayout : MemoryLayout
+ModifiedDate : DateTime
+Name : String
+Number : Int32
+ParameterModified: DateTime
+ProgrammingLanguage : ProgrammingLanguage
+StructureModified : DateTime
Services
+ICompilable
+Export()
+ShowInEditor()
+Delete()
DataBlock
Specific attributes
+IsOnlyStoredInLoadMemory : Boolean
+IsWriteProtectedInAS : Boolean
ArrayDB
Specific attributes
+ArrayDataType : String
+ArrayLimitUpperBound : Int32
GlobalDB
Specific attributes
+DownloadWithoutReinit : Boolean
+IsPLCDB : Boolean
+IsRetainMemResEnabled : Boolean
+RetainMemoryReserve : UInt32 /plus/
InstanceDB
(Note: The differences between the specific attributes of IDBofFB and IDBofUDT will be handled by
General attributes
+InstanceOfName : String
+OfSystemLibElement : String
+OfSystemLibVersion : String
Specific attributes
+DownloadWithoutReinit : Boolean
+InstanceOfNumber : Int32
+InstanceOfType : BlockType
+IsRetainMemResEnabled : Boolean
+MemoryReserve : UInt32 /plus/
+RetainMemoryReserve : UInt32 /plus/
+UDAEnableTagReadback : Boolean
+UDABlockProperties : String



Representation of block diagram

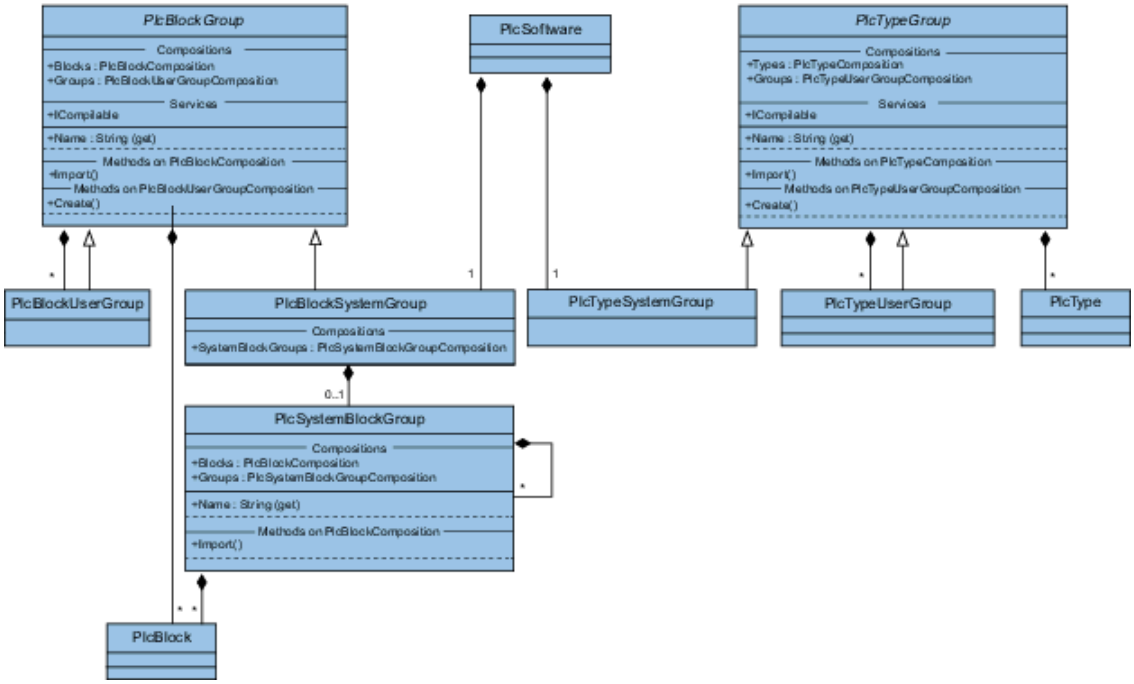
Code and Type



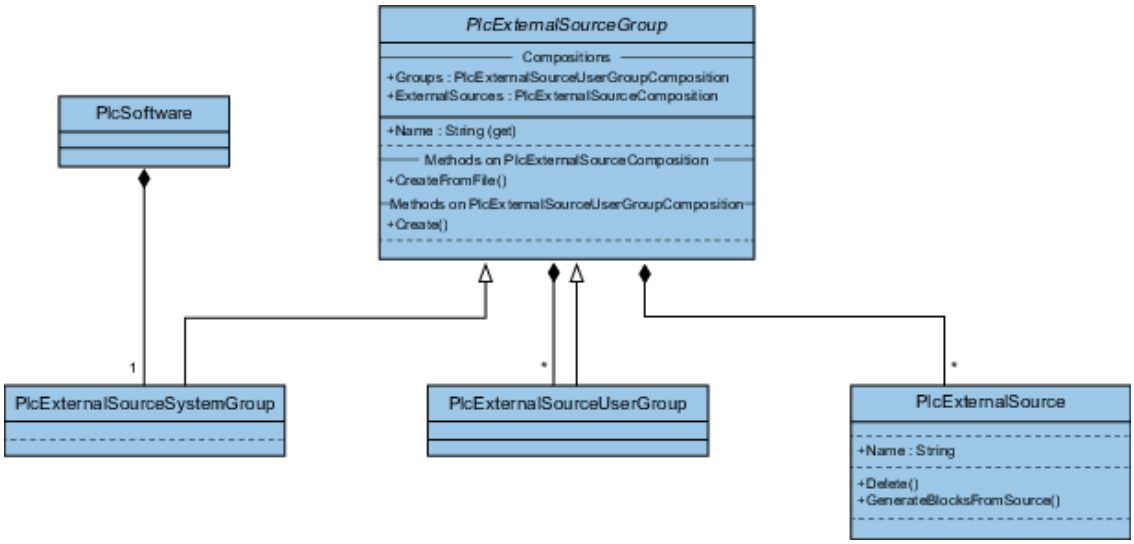
Representation of groups for blocks and types in the TIA Portal Openness API

The two high level groups "PlcBlocks" ("Program blocks" in the GUI of TIA Portal) and "PlcTypes" ("Plc data types" in the GUI of TIA Portal) contain blocks and type definitions. These groups provide the import and the compile functions for blocks. Due to the fact that most of the methods of functionalities of the groups are only achievable via collections, there is an "embedded" or "compacted" representation of the collections and their methods at the "host" classes.

Blocks and Types



External Sources



See also

TIA Portal Openness object model
Hierarchy of hardware objects of the object model

1.1.3 Hierarchy of hardware objects of the object model

Relation between the visible elements in the TIA Portal and the modeled elements in the object model

Hardware object	Explanation
Device (Device)	The container object for a central or distributed configuration.
Device item (DeviceItem)	Each device item object has a container object. The logical relation is "Items".

To understand the hierarchy relationship of hardware objects between what is displayed in the TIA Portal view and what is modeled in the TIA Portal Openness, it is necessary to consider some aspects of the hierarchy of hardware objects.

Example: A HardwareObject has the items link. The Device extends the HardwareObject and has the UnpluggedItems link. The Device is the container object for a station.

The DeviceItem extends the HardwareObject and has the Container link. Every plugged DeviceItem object has got a container object which is a HardwareObject. The reverse relation is Items.

An unplugged DeviceItem object that does not have a container is found via the link UnpluggedItems of the Device.

The container relation is comparable to the relation of the modules for the device item objects.

Example: A device includes one or more racks. A rack includes modules. A module includes submodules.

This is the relation similar to the representation in the network view and device view of the TIA Portal. The "PositionNumber" attribute of a device item is unique in the items area, within a container.

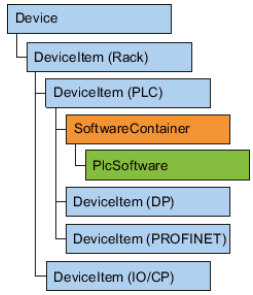
The parent-child relation between device item objects is a purely logical relation in the object model. A child cannot exist without its parents.

- If a submodule is modeled as part of a module (child), the submodule cannot be removed without the module.
- A module or submodule that can be added and removed is a child of the device object

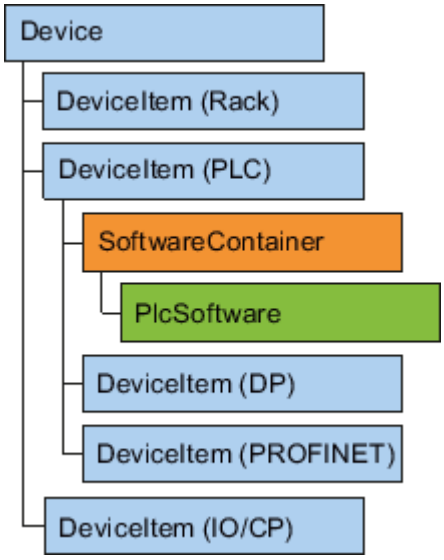
Hierarchy relationships of PLC devices

The diagrams below show the hierarchy relationship between devices, and device items of PLC and HMI devices.

Hierarchy when iterating via .Items	

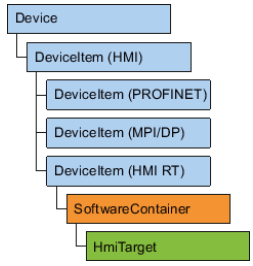


Hierarchy when iterating via .DeviceItems

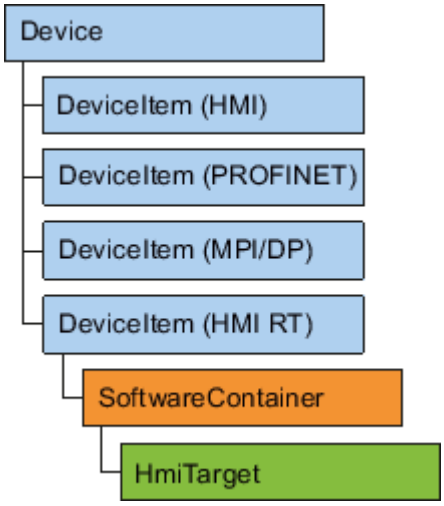


Hierarchy relationships of HMI devices

Hierarchy when iterating via .Items



Hierarchy when iterating via .DeviceItems



See also

TIA Portal Openness object model
Blocks and types of the TIA Portal Openness object model

1.1.4 Object list

Introduction

The following tables show the available objects up to and including Runtime Advanced, and indicate whether these objects are supported by TIA Portal Openness.

Neither Runtime Professional nor device proxy files are supported by TIA Portal Openness in WinCC.

Objects

You can control the following project data depending on which HMI device you are using:

Screens

Object	Basic Panels	Comfort Panels	Mobile Panels	RT Advanced
Screen	Yes	Yes	Yes	Yes
Global screen	Yes	Yes	Yes	Yes
Templates	Yes	Yes	Yes	Yes
Permanent area	No	Yes	Yes	Yes
Pop-up screen	No	Yes	Yes	Yes
Slide-in screen	No	Yes	Yes	Yes

Screen objects

Object	Basic Panels	Comfort Panels	Mobile Panels	RT Advanced
Line	Yes	Yes	Yes	Yes
Polyline	No	Yes	Yes	Yes
Polygon	No	Yes	Yes	Yes
Ellipse	Yes	Yes	Yes	Yes
Ellipse segment	No	No	No	No
Circle segment	No	No	No	No
Elliptical arc	No	No	No	No

Camera view	No	No	No	No
Circular arc	No	No	No	No
Circle	Yes	Yes	Yes	Yes
PDF view	No	No	No	No
Rectangle	Yes	Yes	Yes	Yes
Connector	No	No	No	No
Text field	Yes	Yes	Yes	Yes
Graphic view	Yes	Yes	Yes	Yes
Pipe	No	No	No	No
Double T-piece	No	No	No	No
T-piece	No	No	No	No
Pipe bends	No	No	No	No
I/O field	Yes	Yes	Yes	Yes
Date/time field	Yes	Yes	Yes	Yes
Graphic I/O field	Yes	Yes	Yes	Yes
Editable text field	No	No	No	No
List box	No	No	No	No
Combo box	No	No	No	No
Button	Yes	Yes	Yes	Yes
Round button	No	No	No	No
Illuminated button	No	No	Yes	No

Switch	Yes	Yes	Yes	Yes
Symbolic I/O field	Yes	Yes	Yes	Yes
Key-operated switch	No	No	Yes	No
Bar	Yes	Yes	Yes	Yes
Symbol library	No	Yes	Yes	Yes
Slider	No	Yes	Yes	Yes
Scroll bar	No	No	No	No
Check box	No	No	No	No
Option buttons	No	No	No	No
Gauge	No	Yes	Yes	Yes
Clock	No	Yes	Yes	Yes
Memory space view	No	No	No	No
Function keys	Yes	Yes	Yes	Yes
Faceplate instances	No	Yes	Yes	Yes
Screen window	No	No	No	No
User view	Yes	Yes	Yes	Yes
HTML Browser	No	No	No	No
Print job/script diagnostics	No	No	No	No
Recipe view	No	No	No	No
Alarm view	No	No	No	No
Alarm indicator	No	No	No	No

Alarm window	No	No	No	No
Criteria analysis view	No	Yes [1)]	Yes	Yes
ProDiag overview	No	Yes [1)]	Yes	Yes
GRAPH overview	No	Yes [1)]	Yes	Yes
PLC code view	No	Yes [1)]	Yes	Yes
f(x) trend view	No	No	No	No
f(t) trend view	No	No	No	No
Table view	No	No	No	No
Value table	No	No	No	No
Media Player	No	No	No	No
Channel diagnostics	No	No	No	No
WLAN reception	No	No	No	No
Zone name	No	No	No	No
Zone signal	No	No	No	No
Effective range name	No	No	No	No
Effective range name (RFID)	No	No	No	No
Effective range signal	No	No	No	No
Charge condition	No	No	No	No
Handwheel	No	No	Yes	No
Help indicator	No	No	No	No
Sm@rtClient view	No	No	No	No

Status/Force	No	No	No	No
System diagnostic view	No	No	No	No
System diagnostic window	No	No	No	No

1) Only Mobile Panels with the device version greater than 12.0.0.0 support this screen object

Dynamic

Object	Basic Panels	Comfort Panels	Mobile Panels	RT Advanced
Display	Yes	Yes	Yes	Yes
Operability	No	Yes	Yes	Yes
Visibility	Yes	Yes	Yes	Yes
Movements	Yes	Yes	Yes	Yes

Additional objects

Object	Basic Panels	Comfort Panels	Mobile Panels	RT Advanced
Groups	Yes	Yes	Yes	Yes
Soft keys	Yes	Yes	Yes	Yes
Cycles	Yes	Yes	Yes	Yes
VB scripts	No	Yes	Yes	Yes
Function lists	Yes	Yes	Yes	Yes
Project graphics	Yes	Yes	Yes	Yes

Tags

Object	Basic Panels	Comfort Panels	Mobile Panels	RT Advanced

Multiplex tags	Yes	Yes	Yes	Yes
Arrays	Yes	Yes	Yes	Yes
User data types	Yes	Yes	Yes	Yes
Internal	No	Yes	Yes	Yes
Points of use of elementary data types	Yes	Yes	Yes	Yes
Points of use of user data types	Yes	Yes	Yes	Yes
Points of use of arrays	Yes	Yes	Yes	Yes

TIA Portal Openness also supports all value ranges which are supported by the communication drivers.

Connections

TIA Portal Openness supports non-integrated connections that are also supported by the respective HMI devices. You can find additional information in the online help for the TIA Portal under "Process visualization > Controller communication > Device-dependent".

Lists

Object	Basic Panels	Comfort Panels	Mobile Panels	RT Advanced
Text lists	Yes	Yes	Yes	Yes
Graphic lists	Yes	Yes	Yes	Yes

Texts

Object	Basic Panels	Comfort Panels	Mobile Panels	RT Advanced
Multilingual texts	Yes	Yes	Yes	Yes
Formatted texts and their instances	No	Yes	Yes	Yes

1.2 General functions

1.2.1 Introduction

Overview

TIA Portal Openness supports a selection of functions for defined tasks that you can call outside the TIA Portal by means of the TIA Portal Openness API.

Note

If a previous version of TIA Portal Openness is already installed, the current version will be installed side by side.

You are provided with an overview of the typical programming steps in the sections below. You can learn how the individual code sections interact and how to integrate the respective functions into a complete program. You also get an overview of the code components that have to be adapted for each task.

Example program

The individual programming steps are explained using the "Creating API access in a console application" function as an example. You integrate the provided functions in this program code and adapt the respective code components for this task.

Functions

The section below lists the functions for defined tasks that you can call with TIA Portal Openness outside the TIA Portal.

See also

Applications
Object list

1.2.2 Information about installed TIA Portal Openness versions

Each installed TIA Portal version provides information about available, compatible TIA Portal Openness versions in the Windows registry. This enables TIA Portal Openness applications to automatically start or connect to matching TIA Portal instances.

The relevant TIA Portal Openness registry keys are in the following path in the Windows registry:

HKEY_LOCAL_MACHINE\SOFTWARE\Siemens\Automation\Openness

The registry key “Openness” contains the following subkeys and values:

- Version-independent subkey “AllowList” for the TIA Portal Openness firewall (only from TIA Portal V21 onwards)
  - HKEY_LOCAL_MACHINE\SOFTWARE\Siemens\Automation\Openness\AllowList
  - See also TIA Portal Openness firewall
- Subkeys for each installed TIA Portal version: “xx.xx” (technical version number)
  - HKEY_LOCAL_MACHINE\SOFTWARE\Siemens\Automation\Openness\xx.xx
  - The string value “PortalVersion” for a display string of that TIA Portal version (only from TIA Portal V21 onwards)
- Subkey “PublicAPI”
  - HKEY_LOCAL_MACHINE\SOFTWARE\Siemens\Automation\Openness\xx.xx\PublicAPI
- Subkeys for each provided TIA Portal Openness version in that specific TIA Portal version: “xx.xx.xx.xx” (technical version number)
  - HKEY_LOCAL_MACHINE\SOFTWARE\Siemens\Automation\Openness\xx.xx\PublicAPI\xx.xx.xx.xx
  - The string value “EngineeringVersion” for a display string of that TIA Portal Openness version
- Subkeys for each supported .NET target framework version in that specific TIA Portal Openness version using the Microsoft Target Framework Moniker (TFM) which is a standardized token format for specifying the target framework of a .NET library (only from TIA Portal V21 onwards): “netxxx”
  - HKEY_LOCAL_MACHINE\SOFTWARE\Siemens\Automation\Openness\xx.xx\PublicAPI\xx.xx.xx.xx\netxx
  - The string value “AssemblyVersion” which the Microsoft CLR uses to load the exact version
  - The string value “PublicKeyToken” as part of the strong-named assembly to create a unique identity to prevent assembly conflicts
  - The string value “Siemens.Engineering.Base” pointing to the custom installation path

Note

Regardless which TIA Portal products are installed, the Windows registry only provides the installation path to the base assembly “Siemens.Engineering.Base”.

See also

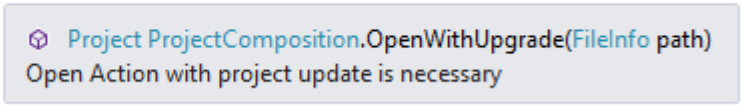
Connecting to the TIA Portal

1.2.3 TIA Portal Openness IntelliSense support

Application

The Intellisense support of TIA Portal Openness helps you at available attributes or methods via tooltip information. It could contain information about the number, names and types of the required parameters. At the following example the bold parameter in the first line indicates the next parameter that is required as you type the function.

```
project = tiaPortal.Projects.OpenWithUpgrade(projectInfo);
```



You can manually invoke Parameter Info by clicking Edit IntelliSense/Parameter Info, typing CTRL+SHIFT+SPACE, or clicking the Parameter Info button on the editor toolbar.

1.2.4 Standard libraries

Insert the following namespace statements at the beginning of the relevant code example to ensure that the code examples work:

```
Copies the following program code to clipboard.

using System;
using System.Collections.Generic;
using System.IO;
using Siemens.Engineering;
using Siemens.Engineering.Cax;
using Siemens.Engineering.Compiler;
using Siemens.Engineering.Compare;
using Siemens.Engineering.Download;
using Siemens.Engineering.Hmi;
using Siemens.Engineering.Hmi.Cycle;
using Siemens.Engineering.Hmi.Communication;
using Siemens.Engineering.Hmi.Globalization;
using Siemens.Engineering.Hmi.RuntimeScripting;
using Siemens.Engineering.Hmi.Screen;
using Siemens.Engineering.Hmi.Tag;
using Siemens.Engineering.Hmi.TextGraphicList;
using Siemens.Engineering.HW;
using Siemens.Engineering.HW.Extensions;
using Siemens.Engineering.HW.Features;
```

```
using Siemens.Engineering.HW.Utilities;
using Siemens.Engineering.Library;
using Siemens.Engineering.Library.MasterCopies;
using Siemens.Engineering.Library.Types;
using Siemens.Engineering.SW;
using Siemens.Engineering.SW.Blocks;
using Siemens.Engineering.SW.ExternalSources;
using Siemens.Engineering.SW.Tags;
using Siemens.Engineering.SW.TechnologicalObjects;
using Siemens.Engineering.SW.TechnologicalObjects.Motion;
using Siemens.Engineering.SW.Types;
using Siemens.Engineering.Upload;
```

1.2.5 Use of the code examples

Structure of the code-snippets

Each code-snippet in this documentation is implemented as a function without return value with an object reference as transfer parameter. Disposing of objects is omitted for the sake of readability. Objects of the TIA Portal are addressed by their name using the `Find` method.




```
//Deletes a single screen from a user folder or a system folder
private static void DeleteScreenFromFolder(HmiTarget hmiTarget)
{
    //The screen "MyScreen" will be deleted if it is existing in the folder "myScreenFolder".
    //If "myScreen" is stored in a subfolder of "myScreenFolder" it will not be deleted.
    string screenName = "MyScreen";
    ScreenUserFolder folder = hmiTarget.ScreenFolder.Folders.Find("myScreenFolder");
    ScreenComposition screens = folder.Screens;
```

```
    Screen screen = screens.Find(screenName);
    if (screen != null)
    {
        screen.Delete();
    }
}
```

You need the following to execute this code-snippet:

- A WinCC project with an HMI device that includes a group with at least one screen.
- A function that instantiates the HMI device.

Note

When you specify directory paths, use the absolute directory path, for example, "C:/path/file.txt".

Relative directory paths are only allowed in the XML files for import and export, for example, "file.txt" or "C:/path01/.../path02/file.txt".

Example for execution of the code-snippet

Use the following example to execute the code-snippet "DeleteScreenFromFolder" as part of the "Hello TIA" example program:



```
//In the sample program "Hello TIA" replace the function call
//"IterateThroughDevices(project)" by the following functions calls:
    HmiTarget hmiTarget = GetTheFirstHmiTarget(project);
    DeleteScreenFromFolder(hmiTarget);
//Put the following function definitions before or after the
//function definition of "private static void IterateThroughDevices(Project project)":
private static HmiTarget GetTheFirstHmiTarget(Project project)
{
    if (project == null)
    {
```

```
        Console.WriteLine("Project cannot be null");
        throw new ArgumentNullException("project");
    }
    foreach (Device device in project.Devices)
    //This example looks for devices located directly in the
project.
    //Devices which are stored in a subfolder of the project
will not be affected by this example.
    {
        foreach (DeviceItem deviceItem in device.DeviceItems)
        {
            DeviceItem deviceItemToGetService = deviceItem as
DeviceItem;
            SoftwareContainer container =
deviceItemToGetService.GetService<SoftwareContainer>();
            if (container != null)
            {
                HmiTarget hmi = container.Software as
HmiTarget;
                if (hmi != null)
                {
return hmi;
                }
            }
        }
    }
    return null;
}
//Deletes a single screen from a user folder or a system
folder
private static void DeleteScreenFromFolder(HmiTarget
hmiTarget)
{
```

```
    string screenName = "MyScreen";
    ScreenUserFolder folder =
hmiTarget.ScreenFolder.Folders.Find("myScreenFolder");
    ScreenComposition screens = folder.Screens;
    Screen screen = screens.Find(screenName);
    if (screen != null)
    {
        screen.Delete();
    }
}
```

1.2.6 Example program

Application example: Creating API access in an application

The following program code shows basic API access in a Console application.

See also: Connecting to the TIA Portal

Note

Enabling the assembly reference property “Copy Local” is not supported. “Copy Local: True” means copying the references to Siemens.Engineering assemblies to the output directory of the application.

Remedy: Set “Copy Local: False” for every Siemens.Engineering assembly after adding the references to the project and clean the output directory.

 Copies the following program code to clipboard.

```
// Assembly reference: Siemens.Engineering.Base.dll
using Siemens.Engineering;
using Siemens.Engineering.Compiler;
using Siemens.Engineering.HW;
using Siemens.Engineering.HW.Features;

// Assembly reference: Siemens.Engineering.Step7.dll
using Siemens.Engineering.SW;
using Siemens.Engineering.SW.Units;

// System
```

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Reflection;

namespace HelloOpenness
{
internal static class Program
{
private const string OpennessFolder = @"C:\Program
Files\Siemens\Automation\Portal Vxx\PublicAPI\Vxx\netxxx";

static Program()
{
// Register assembly resolver at the earliest stage in static
Program constructor
AppDomain.CurrentDomain.AssemblyResolve += OnAssemblyResolve;
}

private static Assembly OnAssemblyResolve(object sender,
ResolveEventArgs args)
{
AssemblyName requestedAssemblyName = new
AssemblyName(args.Name);

string filePath = Path.Combine(OpennessFolder,
string.Concat(requestedAssemblyName.Name, ".dll"));

// Only respond to requests for TIA Portal Openness; must
ignore missing files
if
(!requestedAssemblyName.Name.StartsWith("Siemens.Engineering.")
|| !File.Exists(filePath))
return null;

Assembly loadedAssembly = Assembly.LoadFrom(filePath);

// Check if details like version match the requested assembly
if (requestedAssemblyName.FullName !=
loadedAssembly.GetName().FullName)
throw new FileNotFoundException("TIA Portal Openness version
```

```
does not match", filePath);

return loadedAssembly;
}

public static void Main()
{
Console.WriteLine("Please wait...");
try
{
HelloOpennessApp app = new HelloOpennessApp();
app.Run();
Console.WriteLine("Done");
}
catch (Exception exception)
{
Console.WriteLine("Error");
Console.WriteLine(exception);
}
Console.ReadLine();
}
}

internal class HelloOpennessApp
{
private const string ProjectsPath = @"C:\Projects";

public void Run()
{
using (TiaPortal portal = new
TiaPortal(TiaPortalMode.WithUserInterface))
{
using (ExclusiveAccess exclusiveAccess =
portal.ExclusiveAccess("Generator is working..."))
{
Project project = portal.Projects.Create(new
DirectoryInfo(ProjectsPath), "Project_1");
Console.WriteLine($"Project has been created:
{project.Path.FullName}");

Device station = project.Devices.CreateWithItem(
```

```
"OrderNumber:6ES7 516-3AP03-0AB0/V4.0", "PLC_1", "S7-1500
Station_1"
);
Console.WriteLine($"Device has been created:
{station.Name}");

PlcSoftware plc = GetPlcSoftware(station);
if (plc == null)
{
Console.WriteLine("No PLC software found in the device.");
}
else
{
PlcUnitProvider softwareUnits =
plc.GetService<PlcUnitProvider>();
if (softwareUnits == null)
{
Console.WriteLine("Software units are not supported for this
PLC or firmware version.");
}
else
{
PlcUnit softwareUnit =
softwareUnits.UnitGroup.Units.Create("Software unit_1");
Console.WriteLine($"Software unit has been created:
{softwareUnit.Name}");
}
}

ICompilable compiler = station.GetService<ICompilable>();
if (compiler == null)
{
Console.WriteLine("Compile service is not available for this
device.");
}
else
{
CompilerResult result = compiler.Compile();
Console.WriteLine($"Compile: {result.WarningCount} warnings,
{result.ErrorCount} errors.");
}
```

```
project.Save();
project.Close();
}
}
}

private static PlcSoftware GetPlcSoftware(HardwareObject
hardwareObject)
{
// Breadth-first traversal of a station
Queue<HardwareObject> queue = new Queue<HardwareObject>();
queue.Enqueue(hardwareObject);

while (queue.Count > 0)
{
// Check all modules at the current level
foreach (DeviceItem deviceItem in queue.Dequeue().Items)
{
if (!deviceItem.IsPlugged) continue; // Skip if module is in
parking lot

if
(deviceItem.Classification.HasFlag(DeviceItemClassifications.CPU)
&&
deviceItem.GetService<SoftwareContainer>()?.Software is
PlcSoftware plcSoftware)
return plcSoftware;

// Enqueue all child modules for next level processing
queue.Enqueue(deviceItem);
}
}

return null;
}
}
}
```

1.2.7 Programming steps

Overview

TIA Portal Openness requires the following programming steps for access by means of the TIA Portal Openness API:

1. Make the TIA Portal known in the development environment
2. Set up program access to the TIA Portal
3. Activate program access to the TIA Portal
4. Publish and start the TIA Portal
5. Open project
6. Execute commands
7. Save and close the project
8. Terminate the connection to the TIA Portal

Note

Permitted strings

Only certain characters are allowed in strings in the TIA Portal. All strings passed to the TIA Portal via the TIA Portal Openness application are subject to these rules. If you pass an invalid character in a parameter, an exception is thrown.

See also

Example program
Use of the code examples

1.2.8 Connecting to the TIA Portal

Introduction

TIA Portal Openness has the possibilities to

- get running TIA Portal processes and attach to TIA Portal instances
- start new TIA Portal instances, either with or without graphical user interface

Note

Since you always use TIA Portal Openness by loading Siemens.Engineering assemblies from a specific TIA Portal version, the returned processes are all from that same TIA Portal version. It is not supported to access different versions in the same AppDomain.

The app.config file and the AppDomain.CurrentDomain.AssemblyResolve event are two solutions for resolving the Siemens.Engineering assemblies during runtime.

Note

Resolving Siemens.Engineering assemblies is only needed for TIA Portal Openness applications, but not for TIA Portal Add-Ins.

Using the app.config file

Set “Copy Local: False” for every Siemens.Engineering assembly after adding the references to the project and clean the output directory.

By adding an application configuration file (app.config file) to a .NET project, you can customize how the Microsoft CLR locates and loads assemblies. The app.config file is deployed next to the executable and renamed to match the executable.

Note

The disadvantage of the app.config file is that it must contain the correct entries before the application starts. Changes cannot be made during program execution.

Example app.config file for .NET Framework 4.8:



```
<?xml version="1.0" encoding="utf-8" ?>
<configuration>
<startup>
<supportedRuntime version="v4.0"
sku=".NETFramework,Version=v4.8" />
</startup>
<runtime>
<assemblyBinding xmlns="urn:schemas-microsoft-com:asm.v1">
<dependentAssembly>
<assemblyIdentity name="Siemens.Engineering.Base"
culture="neutral" publicKeyToken="29bfe5fdf4ba5d3b " />
<!-- Edit the following version and path according to your
required TIA Portal Openness version -->
<codeBase version="xx.xx.xx.xx"
href="FILE://C:\Program Files\Siemens\Automation\Portal
Vxx\PublicAPI\Vxx\net48\Siemens.Engineering.Base.dll" />
</dependentAssembly>
<!-- Depending on your application, you may need to provide
additional Siemens.Engineering assemblies -->
```

```
<dependentAssembly>
<assemblyIdentity name="Siemens.Engineering.Base"
culture="neutral" publicKeyToken="29bfe5fdf4ba5d3b " />
<!-- Edit the following version and path according to your
required TIA Portal Openness version -->
<codeBase version="xx.xx.xx.xx"
href="FILE://C:\Program Files\Siemens\Automation\Portal
Vxx\PublicAPI\Vxx\net48\Siemens.Engineering.Base.dll" />
</dependentAssembly>
<!-- Depending on your application, you may need to provide
additional Siemens.Engineering assemblies -->
<dependentAssembly>
<assemblyIdentity name="Siemens.Engineering.Step7"
culture="neutral" publicKeyToken="29bfe5fdf4ba5d3b" />
<!-- Edit the following version and path according to your
required TIA Portal Openness version -->
<codeBase version="xx.xx.xx.xx"
href="FILE://C:\Program Files\Siemens\Automation\Portal
Vxx\PublicAPI\Vxx\net48\Siemens.Engineering.Step7.dll" />
</dependentAssembly>
</assemblyBinding>
</runtime>
</configuration>
```

Using AppDomain.CurrentDomain.AssemblyResolve event

The assembly resolver must be registered at the earliest stage in the program.

Register the AssemblyResolve event before using any TIA Portal Openness types. This cannot happen in the same method. Place all TIA Portal Openness code in a separate method or separate class that is called only after event subscription.

Note: The .NET runtime will throw an exception if it encounters TIA Portal Openness types before the resolver is registered.

Important: This applies not only to method bodies but also to:

- Class properties with TIA Portal Openness types
- Method parameters with TIA Portal Openness types
- Return values with TIA Portal Openness types

Even when placed in separate methods, these will be resolved during class loading.

When you move your TIA Portal Openness code outside the Program class, the resolver will be active before that classes are evaluated during runtime.

Example of recommended application code structure:

 Copies the following program code to clipboard.

```
internal static class Program
{
static Program()
{
// First register assembly resolver at the earliest stage in
static Program constructor
AppDomain.CurrentDomain.AssemblyResolve += OnAssemblyResolve;
}

public static void Main()
{
// Then call method with TIA Portal Openness code in separate
class
HelloOpennessApp app = new HelloOpennessApp();
app.Run();
}

private static Assembly OnAssemblyResolve(object sender,
ResolveEventArgs args)
{
// Code is shown in the next section
// ...
}
}

internal static class HelloOpennessApp
{
public void Run()
{
using (TiaPortal portal = new
TiaPortal(TiaPortalMode.WithUserInterface))
{
// Do something with TIA Portal Openness
}
}
}
```

Example for a basic AssemblyResolve implementation where the installation path is given:

 Copies the following program code to clipboard.

```
private const string OpennessFolder = @"C:\Program
Files\Siemens\Automation\Portal Vxx\PublicAPI\Vxx\netxxx";

private static Assembly OnAssemblyResolve(object sender,
ResolveEventArgs args)
{
AssemblyName requestedAssemblyName = new
AssemblyName(args.Name);

string filePath = Path.Combine(OpennessFolder,
string.Concat(requestedAssemblyName.Name, ".dll"));

// Only respond to requests for TIA Portal Openness; must
ignore missing files
if
(!requestedAssemblyName.Name.StartsWith("Siemens.Engineering.")
|| !File.Exists(filePath))
return null;

Assembly loadedAssembly = Assembly.LoadFrom(filePath);

// Check if details like version match the requested assembly
if (requestedAssemblyName.FullName !=
loadedAssembly.GetName().FullName)
throw new FileNotFoundException("TIA Portal Openness version
does not match", filePath);

return loadedAssembly;
}
```

Note

These examples have demonstrated how to resolve multiple Siemens.Engineering assemblies using a robust and reliable approach. If your application architecture is very modular and your first class only accesses the TiaPortal class from Siemens.Engineering.Base.dll, it is sufficient to resolve only the Siemens.Engineering.Base.dll in your code. All subsequent Siemens.Engineering assemblies will then be automatically resolved by the Base library.

Note

Recommended way is using the TIA Portal Openness registry keys with the AppDomain.CurrentDomain.AssemblyResolve event.

See also: Information about installed TIA Portal Openness versions

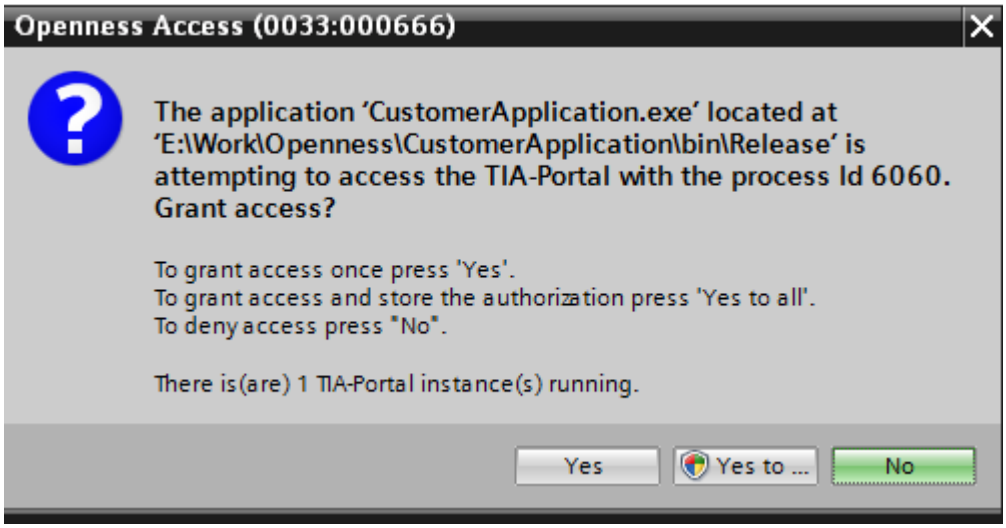
See also

TIA Portal Openness firewall

1.2.9 TIA Portal Openness firewall

TIA Portal Openness firewall prompt

When you try to connect to a running TIA Portal instance (includes starting a new TIA Portal instance) via TIA Portal Openness, TIA Portal will prompt you to accept or reject the connection like the following.



Note

Additionally, and beforehand, administrators must grant individual users the execution of any TIA Portal Openness application in general. This is managed by a local Windows group membership ("Siemens TIA Openness").

See also: Adding users to the "Siemens TIA Openness" user group

Allow connection to the TIA Portal once

Button "Yes": Users can allow individual TIA Portal Openness applications access to a specific (currently active) TIA Portal instance temporarily.

The next time the TIA Portal Openness application tries to connect to a TIA Portal instance, the firewall prompt will be shown again.

Addition of an AllowList entry by connecting to TIA Portal

Button "Yes to all": Administrators can allow individual TIA Portal Openness applications access to every TIA Portal instance permanently. This is managed by an automatically created Windows registry entry.

Note

If the file path, the modification date or the file content (file hash) of the TIA Portal Openness application changes, the application will be identified as a different one and the firewall prompt will be shown again.

Addition of an AllowList entry without using TIA Portal

Administrators can roll out a custom Windows registry file to automatically create an AllowList entry:



```
Windows Registry Editor Version 5.00
[HKEY_LOCAL_MACHINE\SOFTWARE\Siemens\Automation\Openness\AllowList\Custo
[HKEY_LOCAL_MACHINE\SOFTWARE\Siemens\Automation\Openness\AllowList\Custo
"Path"="C:\\Program Files\\CustomerApplication\\bin\\CustomerApplicatior
"DateModified"="2025/06/10 15:09:44.406"
"FileHash"="0rXRKUCNzMWHOMFrT52OwXzqJef10ran4UykTeBraaY="
```

The following example shows how you can calculate the values programmatically:



```
using System;
using System.Globalization;
using System.IO;
using System.Security.Cryptography;

// Usage:
// CalculateValues(new FileInfo("C:\\Program
Files\\CustomerApplication\\bin\\CustomerApplication.exe"));

public static void CalculateValues(FileInfo
opennessApplicationInfo)
{
using (FileStream stream =
opennessApplicationInfo.OpenRead())
{
```

```
HashAlgorithm hashAlgorithm = SHA256.Create();

using (hashAlgorithm)
{
byte[] computedHash = hashAlgorithm.ComputeHash(stream);
DateTime lastWriteTimeUtc =
opennessApplicationInfo.LastWriteTimeUtc;

// This is how the path should be represented
string path = opennessApplicationInfo.FullName;

// This is how the date modified should be formatted
string dateModified = lastWriteTimeUtc.ToString("yyyy/MM/dd
HH:mm:ss.fff", CultureInfo.InvariantCulture);

// This is how the file hash should be represented
string fileHash =
System.Convert.ToBase64String(computedHash);
}
}
}
```

See also: Information about installed TIA Portal Openness versions

1.2.10 Event handlers

Event handlers in TIA Portal Openness application

An instance of the TIA Portal provides the following events to which you can react with an event handler in a TIA Portal Openness application. You can access the attributes of notifications and define the responses accordingly.

Event	Response
Disposed	Use this event to respond to the closing of the TIA Portal with a TIA Portal Openness application.
Notification	Use this event to respond to notifications of the TIA Portal with a TIA Portal Openness application. Notifications require only an acknowledgment, e.g. "OK".

Confirmation	Use this event to respond to confirmations of the TIA Portal with a TIA Portal Openness application. Confirmations always require a decision, e.g. "Do you want to save the project?".

Program code

Modify the following program code to register event handlers in a TIA Portal Openness application:

 Copies the following program code to clipboard.

```
//Register event handler for Disposed-Event
    ....
        tiaPortal.Disposed +=TiaPortal_Disposed;
....
private static void TiaPortal_Disposed(object sender,
EventArgs e)
{
    ....
}
//Register event handler for Notification-Event
    ....
        tiaPortal.Notification += TiaPortal_Notification;
....
private static void TiaPortal_Notification(object sender,
NotificationEventArgs e)
{
    ....
}
//Register event handler for Confirmation-Event
    ....
        tiaPortal.Confirmation += TiaPortal_Confirmation;
....
```

```
private static void TiaPortal_Confirmation(object sender,
ConfirmationEventArgs e)
{
    ....
}
```

Attributes of TIA Portal notifications

TIA Portal notifications have the following attributes:

Attribute	Description
`Caption`	Returns the name of the confirmation.
`DetailText`	Returns the detail text of the confirmation.
`Icon`	Returns the icon of the confirmation.
`IsHandled`	Returns the confirmation or specifies if it is still pending.
`Text`	Returns the text of the confirmation.

Attributes of confirmations

Confirmations have the following attributes:

Attribute	Description
`Caption`	Returns the name of the confirmation.
`Choices`	Returns the option to acknowledge the confirmation.
`DetailText`	Returns the detail text of the confirmation.
`Icon`	Returns the icon of the confirmation.
`IsHandled`	Returns the confirmation or specifies if it is still pending.
`Result`	Returns the result of the acknowledgment or specifies it.

`Text`	Returns the text of the confirmation.

See also

Program-controlled acknowledgement of dialogs with system events

1.2.11 Program-controlled acknowledgement of dialogs with system events

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.
  See Connecting to the TIA Portal
- A project is open.
  See Opening a project
- Event handlers are registered.
  See Connecting to the TIA Portal

Application

When you operate the TIA Portal with the user interface, dialogs with system events are displayed for some program sequences. You decide how you want to proceed based on these system events.

When the TIA Portal is accessed with a TIA Portal Openness application, these system events must be acknowledged by means of corresponding ".NET" events.

The permitted confirmations are contained in the `Choices` list:

- `Abort`
- `Cancel`
- `Ignore`
- `No`
- `NoToAll`
- `None`
- `OK`
- `Retry`
- `Yes`
- `YesToAll`

The value of `ConfirmationEventArgs.Result` must be one of the above-mentioned entries. Otherwise, an exception is thrown.

Program code

Modify the following program code to notify the project engineer about executed actions of a TIA Portal Openness application:

 Copies the following program code to clipboard.

```
using System;
using Siemens.Engineering;
using Siemens.Engineering.HW;
using Siemens.Engineering.HW.Features;
using Siemens.Engineering.SW;
using Siemens.Engineering.SW.Blocks;
using Siemens.Engineering.SW.ExternalSources;
using Siemens.Engineering.SW.Tags;
using Siemens.Engineering.SW.Types;
using Siemens.Engineering.Hmi;
using HmiTarget = Siemens.Engineering.Hmi.HmiTarget;
using Siemens.Engineering.Hmi.Tag;
using Siemens.Engineering.Hmi.Screen;
using Siemens.Engineering.Hmi.Cycle;
using Siemens.Engineering.Hmi.Communication;
using Siemens.Engineering.Hmi.Globalization;
using Siemens.Engineering.Hmi.TextGraphicList;
using Siemens.Engineering.Hmi.RuntimeScripting;
using System.Collections.Generic;
using Siemens.Engineering.Compiler;
using Siemens.Engineering.Library;
using System.IO;
Namespace ConfirmDialogsWithSystemMessagesProgrammatically
{
class internal program()
{
private static void Main(string[] args)
{
```

```
// To respond to a confirmation event
using (TiaPortal tiaPortal = new TiaPortal());
{
tiaPortal.Confirmation += TiaPortalConfirmation(object
sender, ConfirmationEventArgs e);
tiaPortal.Notification += Notification;
try
{
//perform actions that will result in a notification event
}
finally
{
tiaPortal.Notification -= Notification;
}
}
private void TiaPortalConfirmation(object sender,
ConfirmationEventArgs e)
{
    ...
}
//Handles notifications
using (TiaPortal tiaPortal = new TiaPortal())
{
tiaPortal.Notification += Notification;
try
{
//perform actions that will result in a notification event
}
finally
{
tiaPortal.Notification -= Notification;
}
```

```
}
}
}
```

1.2.12 Terminating the connection to the TIA Portal

Introduction

If you started the TIA Portal instance without a user interface and if your application is the only TIA Portal Openness client attached to the TIA Portal, you can close the TIA Portal instance with the TIA Portal Openness application. Otherwise, you disconnect the TIA Portal Openness application from the TIA Portal instance.

Use the `IDisposable.Dispose()` method to separate or close the active instance of the TIA Portal.

You can use the `IDisposable.Dispose()` method as follows:

- With a using statement.
- Surround the object description with a try-finally block and call the `IDispose.Dispose()` method within the finally block.

You can no longer access the TIA Portal when you close the active instance of the TIA Portal.

Note

When a configuration engineer closes the TIA Portal instance despite ongoing access of a TIA Portal Openness application, an exception of the class "NonRecoverableException" is thrown in the TIA Portal Openness application on the next API access. You can subscribe to the dispose event to get a call when the TIA Portal is closed.

Program code

Modify the following program code to separate or close the connection to the TIA Portal:

```
Copies the following program code to clipboard.
// Add code to dispose the application if the application is still instantiated
if (tiaPortal != null)
{
    tiaPortal.Dispose();
}
```

See also

Event handlers

1.2.13 Diagnostic interfaces on TIA Portal

Application

You can retrieve certain diagnostic information from running instances of TIA Portal via a static method. The diagnostic interface is implemented on the TiaPortalProcess object, which can be retrieved for any currently running instance of the TIA Portal.

The diagnostic interface is not blocking, so you can retrieve the TiaPortalProcess object and access it's members, regardless of if the TIA Portal is busy or not. The diagnostic interface includes the following Members:

Class TiaPortalProcess

Member	Type	Function
AcquisitionTime	DateTime	The time when the TiaPortalProcess object was acquired. Since the TiaPortalProcess object represents a completely static snapshot of the state of the TIA Portal at a given point in time, the information it contains may become outdated.
Attach	TiaPortal	Attaches to the given TiaPortalProcess, it returns a TiaPortal instance.
AttachedSessions	IList<TiaPortalSession>	A collection of all other sessions currently attached to the same TIA Portal. This collection can be empty. Each session is represented by a TiaPortalSession object.
Attaching	EventHandler<AttachingEventArgs>	This event enables an application to approve any attempts to attach to the TIA Portal. When another

		application attempts to attach to the TIA Portal, the subscribers of this event are notified and given 10 seconds to approve the attachment. If any subscriber ignores this event or does not respond in time, it is understood to be denying the other application permission to attach. Crashed applications, being unable to respond to this event and cannot cause an application to be denied permission to attach.
Dispose	void	Closes the associated TIA Portal instance.
Id	int	The Process ID of the TIA Portal.
InstalledSoftware	IList<TiaPortalProduct>	A collection of all the products currently installed as part of the TIA Portal. Each product is represented by a TiaPortalProduct object, which is described below.
Mode	TiaPortalMode	The mode in which the TIA Portal was started. The current values are WithUserInterface and WithoutUserInterface.
Path	FileInfo	The path to the executable of the TIA Portal.

ProjectPath	FileInfo	The path to the project which is currently open in the TIA Portal. If no project is open, this attribute will be null.

Class TiaPortalSession

Member	Type	Function
AccessLevel	TiaPortalAccessLevel	The level access of the session. It is represented as a flags enum, where multiple levels of access are possible. TiaPortalAccessLevel is described in detail below.
AttachTime	DateTime	The time when the connection to the TIA Portal was established.
Id	int	The Id of the current session.
IsActive	bool	Returns "true" if the TIA Portal is currently processing a call from the running session.
Dispose	void	Severs the process's connection to the TIA Portal. This method does not kill the process itself in the way that System.Diagnostics.Process.Kill would do. The application whose connection is terminated will still get a disposed event, but there is no other indication of why the connection was terminated.

ProcessId	int	The process ID of the attached process.
ProcessPath	FileInfo	The path to the executable of the attached process.
TrustAuthority	TiaPortalTrustAuthority	Indicates if the current session was started by a process that was signed, and if it is a TIA Portal Openness certificate or not. TrustAuthority is a flags enum and is described below.
UtilizationTime	TimeSpan	The period of time the process has spent actively using the TIA Portal.Combined with the AttachTime attribute, this could be used to determine usage percentages or similar data.
Version	string	The version of TIA Portal Openness to which the session is attached to.

Enum TiaPortalAccessLevel

Enum Value	Function
None	This is not a valid value. It is included because TiaPortalAccessLevel is a flags enum which needs an appropriate "zero value" to represent no flags being set, but it will never appear in actual use because no session can be started that has no access.
Published	The session has access to published functionality.
Modify	The session has modify access.

Enum TiaPortalTrustAuthority

Enum Value	Function
None	The main module of the attached process is not signed with a certificate.
Signed	The main module is signed with a certificate, which is not a TIA Portal Openness certificate.
Certified	The main module is signed with a TIA Portal Openness certificate.
CertifiedWithExpiration	The main module is signed with a TIA Portal Openness certificate that will become invalid at the end of its lifetime.

Class TiaPortalProduct

Member	Type	Function
Name	string	The name of the product (e.g. STEP 7 Professional).
Options	IList<TiaPortalProduct>	A collection of all optional packages that belong to the connected TIA Portal, represented as TiaPortalProduct objects. If an option package itself has option packages, this nesting could continue.
Version	string	The version string of the product.

The following code snippet provides an example of how to use the diagnostic Interface to query information and of how to use them in your application.



```
public void TiaPortalDiagnostics()
{
```

```
    IList<TiaPortalProcess> tiaPortalProcesses =
TiaPortal.GetProcesses();
    foreach (TiaPortalProcess tiaPortalProcess in tiaPortalProcesses)
    {
        Console.WriteLine("Process ID: {0}", tiaPortalProcess.Id);
        Console.WriteLine("Path: {0}", tiaPortalProcess.Path);
        Console.WriteLine("Project: {0}",
tiaPortalProcess.ProjectPath);
        Console.WriteLine("Timestamp: {0}",
tiaPortalProcess.AcquisitionTime);
        Console.WriteLine("UI Mode: {0}", tiaPortalProcess.Mode);
        //See method body below.
        Console.WriteLine("Installed Software:");
        EnumerateInstalledProducts(tiaPortalProcess.InstalledSoftware);
        Console.WriteLine("Attached Openness Applications:");
        foreach (TiaPortalSession session in
tiaPortalProcess.AttachedSessions)
        {
            Console.WriteLine("Process: {0}", session.ProcessPath);
            Console.WriteLine("Process ID: {0}", session.ProcessId);
            DateTime attachTime = session.AttachTime;
            TimeSpan timeSpentAttached = DateTime.Now - attachTime;
            TimeSpan utilizationTime = session.UtilizationTime;
            long percentageTimeUsed = (utilizationTime.Ticks /
timeSpentAttached.Ticks) * 100;
            Console.WriteLine("AttachTime: {0}", attachTime);
            Console.WriteLine("Utilization Time: {0}",
utilizationTime);
            Console.WriteLine("Time spent attached: {0}",
timeSpentAttached);
            Console.WriteLine("Percentage of attached time spent using
TIA Portal: {0}", percentageTimeUsed);
            Console.WriteLine("AccessLevel: {0}", session.AccessLevel);
```

```
            Console.WriteLine("TrustAuthority: {0}",
session.TrustAuthority);
            if ((session.TrustAuthority &
TiaPortalTrustAuthority.Certified) !=
TiaPortalTrustAuthority.Certified)
            {
                Console.WriteLine("TrustAuthority doesn't
match required level, attempting to terminate connection to
TIA Portal.");
                session.Dispose();
            }
        }
    }
}
public void
EnumerateInstalledProducts(IEnumerable<TiaPortalProduct>
products)
{
    foreach (TiaPortalProduct product in products)
    {
        Console.WriteLine("Name: {0}", product.Name);
        Console.WriteLine("Version: {0}", product.Version);
        //recursively enumerate all option packages
        Console.WriteLine("Option Packages \n:");
        EnumerateInstalledProducts(product.Options);
    }
}
```

Security Relevant Information

Because of the fact that no connection to the TIA Portal is needed to use the diagnostics interface, it's possible to write a Windows service that uses the attaching event to check any application attempting to attach to a TIA Portal, e.g. only applications that begin with your company's name are allowed to attach. Another option might be to always grant access, but write information about attaching processes to a log. The following program code is an example event handler to check incoming connections:

 Copies the following program code to clipboard.

```
public void OnAttaching(object sender, AttachingEventArgs e)
{
    string name =
Path.GetFileNameWithoutExtension(e.ProcessPath);
    TiaPortalAccessLevel requestedAccessLevel = e.AccessLevel
& TiaPortalAccessLevel.Published;
    TiaPortalTrustAuthority certificateStatus =
e.TrustAuthority &TiaPortalTrustAuthority.Certified;
    if (requestedAccessLevel ==
TiaPortalAccessLevel.Published &&
       certificateStatus == TiaPortalTrustAuthority.Certified
&&
       name.StartsWith("SampleCustomerName"))
    {
        e.GrantAccess();
    }
}
```

1.2.14 Exclusive access

Requirement

- The TIA Portal Openness application is connected to the TIA Portal.

  See Connecting to the TIA Portal
- A project is open.

  See Opening a project

Application

The "TIA Portal" class provides the method "ExclusiveAccess(String text)" to establish an exclusive access to an attached TIA Portal process. The usage of an exclusive access is highly recommended even if it is not mandatory.

Use "ExclusiveAccess" in an "using" statement to ensure it is disposed attribute even when exceptions occur or the application is shutdown.

Note

Any attempt to create a second exclusive access within the scope of an open exclusive access will result in an recoverable exception being raised.

Modify the following example to get "ExclusiveAccess" to an instance:



```
...
[assembly: AssemblyTitle("MyApplication")]
// This will be used for the exclusive access dialog when present....
TiaPortal tiaPortal = ...;
using (ExclusiveAccess exclusiveAccess = tiaPortal.ExclusiveAccess("My Activity"))
{
    ...
}
```

After acquiring an "ExclusiveAccess" instance for a given TIA Portal process a dialog will be displayed. This dialog will display the message provided during instantiation. In addition the following information of the client application will be displayed:

- the assembly title of the manifest data if available; otherwise, the process name
- the process ID
- the SID (Openness Session ID of the client application's TiaPortal instance)

Note

There can be multiple sessions active for a given TIA Portal Openness client application because there can be multiple instances of TiaPortal each associated with the same TIA Portal process.

The client application can also update the displayed content of the exclusive access dialog by setting the "Text" attribute with new values. Modify the following program code to invoke this behavior:



```
exclusiveAccess = ...;
...
exclusiveAccess.Text = "My Activity Phase 1";
...
```

```
exclusiveAccess.Text = "My Activity Phase 2";
...
exclusiveAccess.Text = String.Empty; // or null;
...
```

You can request that the exclusive access could be cancelled by selecting the "Cancel" button. Modify the following program code to invoke this behavior:

```
exclusiveAccess = ...;
...
if (exclusiveAccess.IsCancellationRequested)
{
    // stop your activity
    ...
}
else
{
    // continue your activity
    ...
}
...
```

1.2.15 Dynamic behaviours

Dynamic behaviors in Openness

The Openness users can determine objects and their attributes during the run time. They also are able to invoke actions and navigate to related objects bidirectionally in the hierarchy. In order to support dynamic behaviors the following interfaces are provided and implemented by Openness objects:

IEngineeringInstance

The interface that allows up navigation in the Object Model.

Properties:

- IEngineeringObject Parent - Parent object of an instance.

IEngineeringCompositionOrObject

The interface is implemented by both engineering objects and compositions.

IEngineeringObject

The interface is implemented by the majority of objects that are available for the Openness users. IEngineeringObject interface is derived from IEngineeringCompositionOrObject and IEngineeringInstance:

Methods:

- IList<EngineeringCreationInfo> GetCreationInfos(string compositionName) - Gets the list of composition infos available for the object.
- IEngineeringObject Create(string compositionName, Type type, IEnumerable<KeyValuePair<string, object>> parameters) - Creates an IEngineeringObject of indicated type initialized with values as indicated compositionName within the parameters
- object GetAttribute(string name) - Gets an attribute with the given name.
- IList<EngineeringAttributeInfo> GetAttributeInfos() - Returns a collection of EngineeringAttributeInfo objects describing the different attributes on this object.
- IList<object> GetAttributes(IEnumerable<string> names) - Gets a list of attributes for the given names
- IEngineeringCompositionOrObject GetComposition(string name) - Gets an IEngineeringCompositionOrObject with the given name.
- IList<EngineeringInvocationInfo> GetInvocationInfos() - Returns a collection of EngineeringInvocationInfo objects describing the different actions on this object.
- object Invoke(string name, IEnumerable<KeyValuePair<Type, object>> parameters) - Invokes the method represented by the current instance, using the specified parameters.
- void SetAttribute(string name, object value) - Sets an attribute with the given name to the given value
- void SetAttributes(IEnumerable<KeyValuePair<string, object>> attributes) - Sets the attributes with the given names to the given values as indicated in attributes.
- EngineeringObjectHandle GetHandle() - Gets the object's unique handle.
- void SetAttributes(IEnumerable<KeyValuePair<string, object>> attributes, AttributeDelegate errorHandler) - Sets the attributes with the given names to the given values as indicated in attribute key value pair. If any error occurs (such as, attribute name/value is invalid), a callback call will be triggered so that you can decide whether to ignore and continue with setting value to the next attribute or abort setting the values to the remaining attributes.

The SetAttributes(IEnumerable<KeyValuePair<string, object>> attributes, AttributeDelegate errorHandler) is supported by the following Engineering objects:

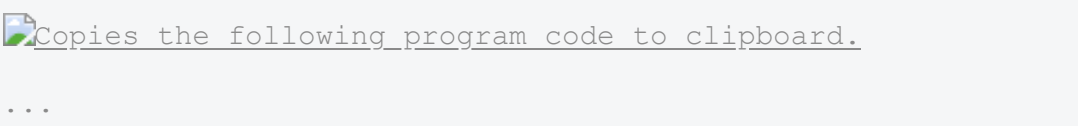
- Siemens.Engineering.HW.Features.NetworkInterface
- Siemens.Engineering.HW.Features.NetworkPort

- Siemens.Engineering.HW.Features.PlcAccessLevelProvider
- Siemens.Engineering.HW.Address
- Siemens.Engineering.HW.Channel
- Siemens.Engineering.HW.Device
- Siemens.Engineering.HW.DeviceImpl
- Siemens.Engineering.HW.DeviceItem
- Siemens.Engineering.HW.DeviceItemImpl
- Siemens.Engineering.HW.HardwareObject
- Siemens.Engineering.HW.IoConnector
- Siemens.Engineering.HW.IoController
- Siemens.Engineering.HW.IoSystem
- Siemens.Engineering.HW.MrpDomain
- Siemens.Engineering.HW.Node
- Siemens.Engineering.HW.Subnet
- Siemens.Engineering.HW.SyncDomain
- Siemens.Engineering.HW.TransferArea

If any other Engineering object is used, the method behaves similar to the method void SetAttributes(IEnumerable<KeyValuePair<string, object>> attributes) which has no callback.

- GetAttributes(Siemens.Engineering.AttributeAccessOption) - Get all attributes of an engineering object filtered by access mode with a single call. You can use the flag enum Siemens.Engineering.AttributeAccessOptions as a parameter for bulk API GetAttributes on an Engineering object to filter attributes based on access modes. The below options are available under AttributeAccessOptions:
  - None: No Attributes will be retrieved with this option. Empty list will be returned if this option is used in GetAttributes API.
  - ReadOnly: Attributes which have only read access.
  - WriteOnly: Attributes which have only write access. An exception of type Siemens.Engineering.EngineeringNotSupported Exception will be thrown If there is at-least one write only attribute on an engineering object and user invokes GetAttributes with AttributeAccessOptions.WriteOnly option.
  - ReadWrite: Attributes which have read and write access.

Program code: SetAttributes(IEnumerable<KeyValuePair<string, object>>) method



```
...
List<KeyValuePair<string, object>> attributeKeyValuePair =
new List<KeyValuePair<string, object>>()
{
new KeyValuePair<string, object>("Name", "DeviceItemName")
};
deviceItem.SetAttributes(attributeKeyValuePair,
(AttributeConfiguration attributeConfiguration) =>
{
//This callback delegate will be invoked when there is an
error while executing the SetAttributes API,
//reason could be either due to the attribute having invalid
attribute name, invalid value,
//invalid type of the value or when tried to set the value to
a readonly attribute
switch (attributeConfiguration)
{
case AttributeReadOnly attributeReadOnly:
attributeReadOnly.CurrentSelection =
AttributeChoiceSelection.Ignore;
break;
case AttributeNameUnsupported attributeNameUnsupported:
//Attribute name is not supported
attributeNameUnsupported.CurrentSelection =
AttributeChoiceSelection.Ignore;
// Or AttributeChoiceSelection.Abort
break;
case AttributeTypeUnsupported attributeTypeUnsupported:
//The the type of supplied value is not supported
attributeTypeUnsupported.CurrentSelection =
AttributeChoiceSelection.Ignore; // Or
AttributeChoiceSelection.Abort
break;
case AttributeValueUnsupported attributeValueUnsupported:
//The value is not supported
```

```
attributeValueUnsupported.CurrentSelection =
AttributeChoiceSelection.Ignore; // Or
AttributeChoiceSelection.Abort
break;
}
}
);
```

Program code: GetAttributes(Siemens.Engineering.AttributeAccessOption)



```
Copies the following program code to clipboard.
private void GetAllAttributes(ProjectBase project)
{
    DeviceItem deviceItem = project.Devices[0].Items[0];
    // If there is no readonly attribute available in
DeviceItem
    // then the returned list will be empty.
    IReadOnlyList<KeyValuePair<string, object>>
attributeNameValuePairs =
deviceItem.GetAttributes(AttributeAccessOptions.ReadOnly);
    foreach(KeyValuePair<string, object> pair in
attributeNameValuePairs)
    {
    Console.WriteLine("Attribute name: {0}, value: {1}",
pair.Key, pair.Value);
    }
}
```

IEngineeringRoot

This interface is implemented by TiaPortal objects and indicates that the user is at the top object in the hierarchy. IEngineeringRoot is derived from IEngineeringObject, IEngineeringCompositionOrObject and IEngineeringInstance.

Methods:

- IEngineeringObject GetObject(EngineeringObjectHandle objectHandle) - Gets the object from a handle

IEngineeringComposition

The interface is implemented by the majority of objects that are available for the Openness users. IEngineeringObject interface is derived from IEngineeringCompositionOrObject and IEngineeringInstance. In addition to the properties and methods described in Working with compositions, the following methods are supported.

Methods:

- IEngineeringObject Create(Type type, IEnumerable<KeyValuePair<string, object>> parameters) - Creates an IEngineeringObject of indicated type initialized with values as indicated in parameters..
- IList<EngineeringCreationInfo> GetCreationInfos() - Gets the collection of EngineeringCreateInfo objects describing the different CreateInfos on this object.
- IList<EngineeringInvocationInfo> GetInvocationInfos() - Returns a collection of EngineeringInvocationInfo objects describing the different actions on this object.
- object Invoke(string name, IEnumerable<KeyValuePair<Type, object>> parameters) - Invokes the method represented by the current instance, using the specified parameters.

IEngineeringObjectAssociation

This interface is implemented by the collection of associated engineering objects. This interface is derived from IEngineeringAssociation and IEngineeringInstance. See Working with associations

IEngineeringServiceProvider

The interface is implemented by engineering objects that can provide service(s).

Methods:

- T GetService<T>() where T : class, IEngineeringService - Gets an instance of type T.
- IList<EngineeringServiceInfo> GetServiceInfos() - Returns a collection of EngineeringServiceInfo objects describing the different services on this object.

IEngineeringService

The interface representing an engineering service that is provided by IEngineeringServiceProvider.

EngineeringObjectHandle

The structure that provides a unique object handle.

See also

Working with compositions
Working with associations

1.2.16 Working with associations

Accessing associations

An association describes the relationship between two or more objects at type level.

TIA Portal Openness supports access to associations via index and via "foreach" loops. Direct access, for example via string name, is not supported.

Attributes

The following attributes are available:

- `int Count`
- `bool IsReadonly`
- `IEngineeringObject Parent`
- `retType this [ int index ] { get; }`

Methods

TIA Portal Openness supports the following methods:

- `int IndexOf ( type )`: Returns the index in the association for a transferred instance.
- `bool Contains ( type )`: Determines whether the transferred instance is contained in the association.
- `IEnumerator GetEnumerator <retType>()`: Employed within "foreach" loops to access an object.
- `void Add ( type )` [1]: Adds the transferred instance to the association.
- `void Remove ( type )` [1]: Removes the transferred instance from the association.

[1]: Not supported by all associations.

1.2.17 Working with compositions

Accessing compositions

A composition is the special case of an association. A composition expresses a semantic relationship of two objects, of which one is part of the other.

Attributes

The following attributes are available:

- `int Count`
- `bool IsReadonly`
- `IEngineeringObject Parent`

- `retType this [int index] {get;}`: Indexed access to an object of the composition.

  This type of access should only be used in a targeted manner, as each indexed access operation exceeds process boundaries.

Methods

TIA Portal Openness supports the following methods:

- `retType Create (id, …)`: Creates a new instance and adds this instance to the composition.

  The signature of the method depends on the way in which the instance is created. This method is not supported by all compositions.

- `type Find (id, …)`: Scans a composition for the instance with the transferred ID.

  The search is not recursive. The signature of the method depends on the way in which the instance is searched for. This method is not supported by all compositions.

- `IEnumerator GetEnumerator<retType> ()`: Employed within "foreach" loops to access an object.

- `Delete (type)` [1]: Deletes the instance specified by the current object reference.

- `int IndexOf (type)`: Returns the index in the composition for a transferred instance.

- `bool Contains (type)`: Determines whether the transferred instance is contained in the composition.

- `void Import(string path, ImportOptions importOptions)` [1]: Used for each composition that contains importable types.

  Each import signature includes a configuration parameter of the type "ImportOptions" ("None", "Overwrite") by which the user controls the import behavior.

[1]: Not supported by all compositions.

1.2.18 Verifying object equality

Application

As user of a TIA Portal Openness API, you can check that objects are the same with program code:

- You check whether two object references are the same with the operator "`==`".
- Use the `System.Object.Equals()` method to check if both objects are really identical with regard to the TIA Portal.

Program code

Modify the following program code to check for object reference types:

 Copies the following program code to clipboard.

```
...
//Composition
DeviceComposition sameCompA = project.Devices;
DeviceComposition sameCompB = project.Devices;
if (sameCompA.Equals(sameCompB))
{
    Console.WriteLine("sameCompA is equal to sameCompB");
}
if (!(sameCompA == sameCompB))
{
    Console.WriteLine("sameCompA is not reference equal to
sameCompB");
}
DeviceComposition sameCompAsA = sameCompA;
if (sameCompAsA.Equals(sameCompA))
{
    Console.WriteLine("sameCompAsA is equal to sameCompA");
}
if (sameCompAsA == sameCompA)
{
    Console.WriteLine("sameCompAsA is reference equal to
sameCompA");
}
MultiLingualGraphicComposition notSameComp =
project.Graphics;
if (!sameCompA.Equals(notSameComp))
{
    Console.WriteLine("sameCompA is not equal to
notSameComp");
}
```

1.2.19 Read operations for attributes

Group operations and standard read operations for attributes

TIA Portal Openness supports access to attributes via the following methods which are available at the object level:

- Group operation for read access
- Standard read operations

Program code for group operations